

Join Go

공간 적응형 로봇 검사 자동화 플랫폼

협동 2 프로젝트 | 2026.05
AI(Computer Vision)기반 협동 로봇 작업
어시스턴트 구현 프로젝트

C-2조

윤성웅 윤재현 조의연 김지홍

강사님 : 이충현

01

프로젝트 개요

로봇 비전 검사와 데이터 기반 품질 추적을 결합한 공간 적응형 검사 시스템

4명

개발 인원

파트별 담당 분리

YOLO

검사 AI

볼트 유무 검출

Firebase

운영 DB

session 기반 저장

RealSense

RGB-D 입력

공간·이미지 데이터

Robot

후속 작업

불량 좌표 활용

개발 흐름

1. 작업공간 촬영 → 2. YOLO 볼트 검출 → 3. 볼트 위치/유무 결과 생성 → 4. session DB 저장 → 5. 웹/로봇/디지털 트윈 확장 데이터 제공

핵심 차별점

- 포인트 클라우드(Point Cloud) 기반의 실시간 3D 공간 맵핑 및 웹 라이브 뷰 동기화
- 일회성 파일 저장을 탈피한 [세션 → 작업대 → 캡처 → 마커] 단위의 관계형 데이터 구조화
- 다중 작업대별 목표 나사 위치 다이내믹 탐색 및 적응형 로봇 제어(Postural & Torque Control)

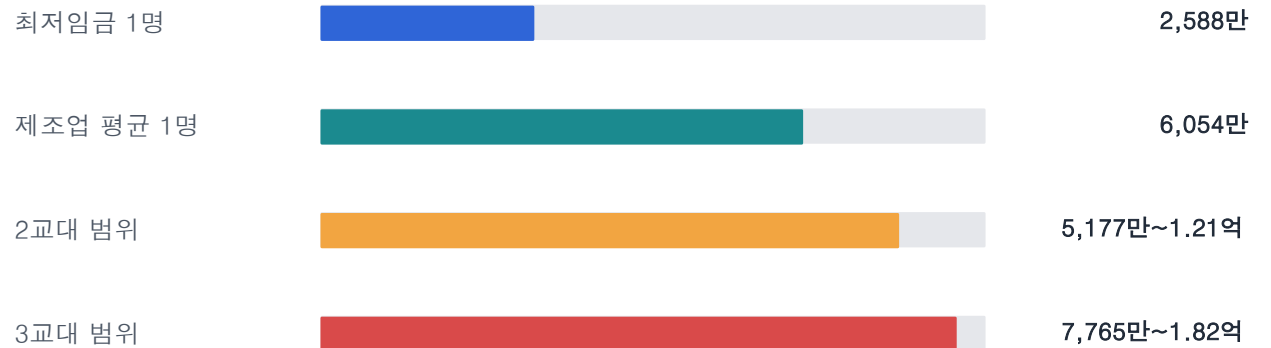
시장 요구사항 및 도입 필요성

반복 육안 검사는 인건비·품질 편차·추적 비용을 동시에 만든다

왜 필요한가

검사자는 매번 같은 위치를 확인해야 하고, 반복 작업은 피로도에 따라 품질 편차를 만든다. 결과가 수기·이미지 중심이면 불량 위치 추적과 후속 조치도 느려진다.

연간 직접 인건비 기준



시장 요구

검사 인력 상시 투입을 줄이고, 같은 기준으로 반복 검사하며, 불량 위치를 데이터로 남길 수 있는 자동화 구조가 필요하다.

출처: 고용노동부 2026 최저임금, Trading Economics 2025 Q3 제조업 임금

기존 문제점 및 솔루션

고정형·수작업 검사 방식의 한계를 로봇 검사와 운영 DB로 해결

기존 문제	현장 영향	해결 구조
반복 검사 인력 부담	인건비와 피로도 증가	로봇·비전 기반 자동 검사
검사 품질 편차	속련도·조명·각도에 따라 판단 흔들림	AI 기준 + 실제 hardcase 재학습
불량 위치 추적 어려움	후속 조치와 원인 분석 지연	marker 좌표·상태 DB 저장
작업공간 변경 대응 어려움	설비 재세팅 비용 증가	session/workstation 기준 재구성
외부 공유 구조 부족	디지털 트윈·API 연동 어려움	index·twin_state·robot pointer 설계

로봇 도입으로 기대되는 개선 범위

실제 수치는 공정 조건에 따라 달라지며, 아래 값은 파일럿 도입 목표 범위

50~70%↓

직접 검사 투입 시간

육안 전수 확인에서 결과 모니터링 중심으로 전환

30~50%↓

불량 위치 확인 시간

웹 화면에서 불량 마커와 좌표를 바로 확인

60~80%↓

보고·정리 작업

검사 결과·수량·불량률을 자동 저장 및 요약

Detection

볼트 검출 구조

상태 분류 제외, 존재 여부만 검출

1-Class

볼트 검출 기준

good/ng 판단 제외, 볼트 유무만 검출

구분	기존 방식	JoinGo 적용 후
검사	작업자 직접 확인	로봇·카메라·AI 검사
기록	수기·이미지 중심	DB 자동 저장
위치	현장 재확인 필요	좌표·마커 기반 확인
조회	전체 결과 탐색	index 포인터 즉시 접근

개선률은 실측 전 목표 범위이며, DB/YOLO 성능 수치는 구현·학습 결과 기준으로 정리

Scan-First 로봇 검사 컨셉

설비에 검사를 맞추는 것이 아니라, 로봇이 먼저 공간을 스캔하고 현재 설비 기준으로 검사를 시작

고정형 검사 설비

설비 기준이 고정됨

카메라 위치·작업대 위치·검사 대상이 바뀌면 기준을 다시 맞춰야 한다.

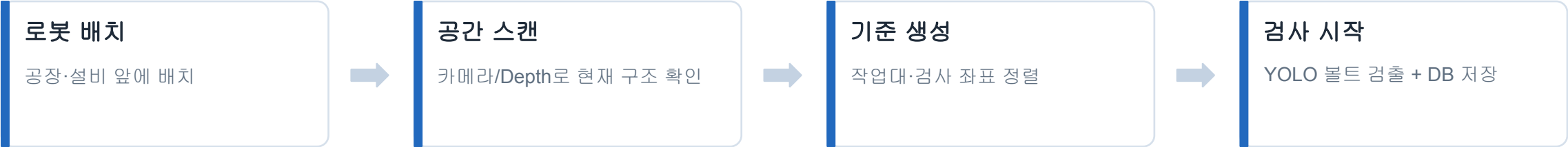
라인 변경이나 설비 이동 시 재설정 부담이 커진다.

JoinGo Scan-First 구조

작업 전 공간을 먼저 스캔

로봇을 배치한 뒤 현재 작업공간을 스캔하고, 설비 위치와 검사 기준을 현재 공간 기준으로 다시 구성한다.

다른 설비에도 같은 검사 흐름을 적용하기 쉽다.



핵심: 검사 기준을 설비에 고정하지 않고, 스캔된 현재 작업공간에 맞춰 생성

어느 설비에 놓아도 확장 가능한 이유

작업공간을 먼저 인식하고, 검사 결과를 session·workstation·capture·marker 단위로 저장하기 때문

1. 공간 재인식

새 작업공간의
위치·형상·검사 대상 배치를
스캔
2. 기준 좌표 설정

카메라/로봇/작업대 기준을
현재 설비에 맞게 정렬
3. 검사 대상 매핑

나사·부품 위치를 marker
단위로 생성하고 상태를
부여
4. 운영 DB 저장

session → workstation →
capture → marker 구조로
누적
5. 외부 연동

디지털 트윈·GUI·로봇
제어가 같은 경로를 조회

구분	고정형 검사 방식	Scan-First 방식
설비 이동	카메라/지그 기준 재설정 필요	스캔 후 현재 설비 기준으로 재매핑
작업대 변경	검사 위치 재등록 부담	workstation 단위로 새 검사 흐름 생성
협업 연동	데이터 문맥 부족	session·capture·marker 경로로 공유 쉬움
확장성	특정 공정 전용에 가까움	다른 공장·설비로 확장 가능

결과: 작업공간 변경 시 “재개발”이 아니라 “재스캔 + 재매핑”으로 대응

팀 구성 및 역할 분담

PART 1을 Data/DB·YOLO Debugging으로 배치하고 4명 역할을 분리

PART 1 Data / DB · YOLO / Final Inspection 윤성웅

- Firestore DB 구조 개선
- session/workstation 설계
- YOLO 학습·검증 디버깅
- 검사 결과 저장 흐름 정리
- 최종 점검 및 피드백

PART 2 Vision / 3D [윤재현]

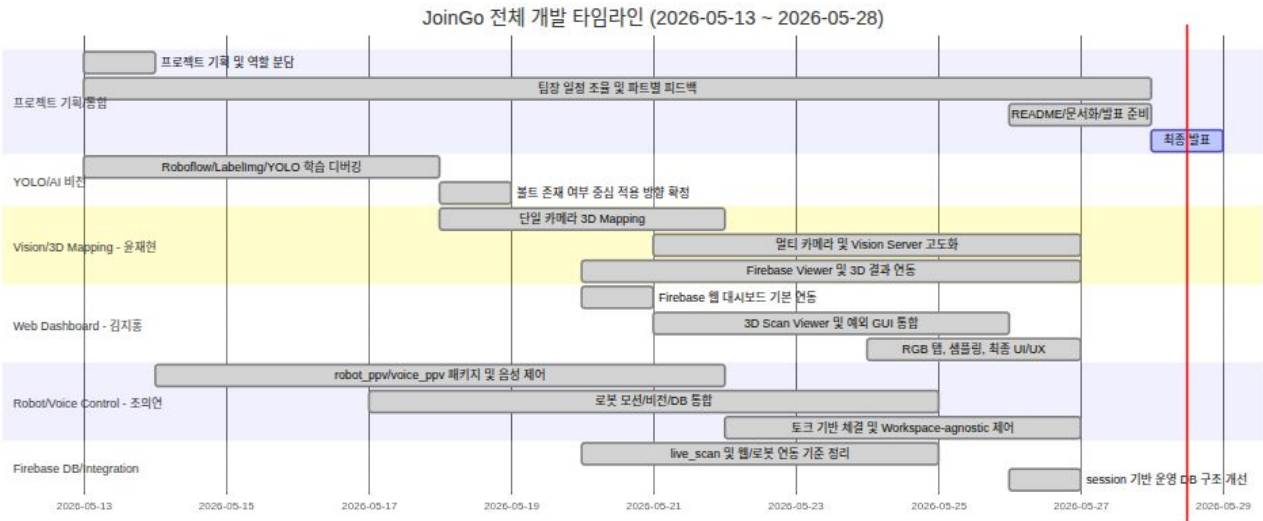
- RealSense 입력
- PointCloud 생성
- 공간 인식·시각화
- 3D 데이터 처리

PART 3 Robot Control [조의연]

- 로봇 제어
- 좌표 변환
- 작업 시퀀스

PART 4 Web [김지훈]

- 관리 화면
- API 연동
- 시연 흐름



시스템 아키텍처 상세

Robot·Vision·DB·Web을 연결하여 검사 결과를 재사용 가능한 품질 데이터로 제공



데이터 수집

- RealSense D435i 카메라 연동
- ROS2 기반 실시간 데이터 스트리밍
- 작업 공간 및 3D PointCloud 획득



전처리 및 분석

- YOLO v11 Bolt Detector
- OpenCV 이미지 처리 및 노이즈 제거
- bbox, class, confidence 데이터 생성



클라우드 전송

- Firebase Admin SDK 통합
- Realtime Database 실시간 동기화
- Storage 기반 고용량 이미지 저장



사용자 인터페이스

- Web Live View 대시보드
- 디지털 트윈 기반 상태 모니터링
- 이력 조회 및 결과 시각화

노드·모듈 아키텍처

센서 입력, AI 판별, DB 저장, 결과 조회를 분리한 모듈 구조



Camera Node

RGB / Depth / CameraInfo 입력



YOLO Node

YOLO Detector → Bolt bbox → Marker



DB Writer

capture_data와 marker 저장



Index Updater

latest / lookup / defects index 갱신

Firebase Realtime Database

inspections / sessions / workstations /
captures / markers
indexes / sites / robots / twin_state

External Consumers

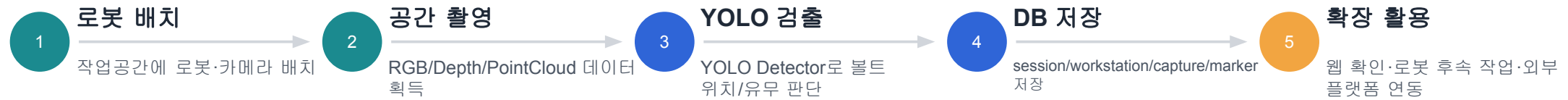
GUI, Robot Control, Digital Twin
Platform, 외부 API가 같은 DB 기준을
읽는다.

설계 의도

모듈별 책임을 나누어 한쪽 구현이 바뀌어도 검사 결과 데이터 구조는 유지되도록
구성한다.

전체 플로우 차트

로봇 배치부터 AI 검사, DB 저장, 웹·디지털 트윈 확장까지의 데이터 흐름



핵심 데이터 흐름

실제 현장 데이터가 AI 판별 결과로 바뀌고, DB 구조를 통해 웹·로봇·디지털 트윈이 공통으로 사용할 수 있는 품질 데이터가 된다.

PART 1 · DB

PART 1 개발 환경 및 기술 스택

Data / Firebase DB / YOLO Debugging — 검사 결과를 추적 가능한 데이터로 만드는 파트

구분	사용 기술	적용된 부분
운영 환경	Ubuntu / 개발 PC	YOLO 학습·검증 및 Firebase 연동 테스트
AI Framework	Ultralytics YOLO	Detection 학습·검증
모델	YOLOv8n, YOLO11x	초기 모델 비교와 최종 단일 검출 구조
라벨링	Labellmg	local YOLO 라벨 검수 및 수정
카메라	Intel RealSense D435i	실시간 RGB 입력 기반 테스트
DB	Firebase Realtime Database	session/workstation/capture/marker 저장
Storage	Firebase Storage	capture 배경·이미지 파일 경로 정합성
검증	py_compile, git diff, import test	DB 구조 전환 중 오류 복구와 검증

다른 회사와 협업할 때의 강점

디지털 트윈·스마트팩토리 회사가 보기 쉬운 현장 검사 데이터를 표준 구조로 제공



제공 데이터	저장 기준	협업 시 활용
검사 세션	session_id	검사 시작·종료 단위 리포트
볼트 위치	marker 위치 + confidence	디지털 트윈 위 볼트 위치 마커 표시
검사 결과	capture_id + summary	최신 결과·날짜별 결과 조회
불량 위치	marker 상태 + 좌표	디지털 트윈 위 불량 마커 표시

핵심 가치: “어느 설비에서 나온 검사 결과인지”를 경로와 metadata로 함께 제공

DB를 왜 강조하는가

검사 자동화의 가치는 “검출 결과를 다른 시스템이 이해할 수 있는 운영 데이터로 바꾸는 것”에서 발생

문제 인식

검출 결과만 있을 때 생기는 한계

결과 고립	이미지 한 장 안의 normal/defect로 끝나면 이후 시스템이 재사용하기 어려움
위치 맥락 부족	어느 site, session, workstation의 결과인지 외부에서 판단하기 어려움
협업 전달 어려움	다른 회사가 데이터를 받을 때 경로·필드 의미를 다시 해석해야 함
로봇 연동 약함	불량 위치를 후속 작업 대상 좌표로 바로 쓰기 어려움

핵심 방향

DB는 단순 저장소가 아니라, Vision·Robot·GUI·외부 회사가 같은 검사 결과를 바라보게 하는 공통 언어로 설계

협업 가치

- 외부 회사가 보기 쉬움**
session / workstation / capture / marker 단위로 경로가 나뉘어 데이터 의미가 명확
- API 공유가 쉬움**
capture_id만 전달해도 lookup index로 원본 경로와 요약 정보 추적 가능
- 품질 리포트 생성**
세션별 총 검사 수·정상 수·불량 수를 바로 집계해 보고서에 사용
- 후속 로봇 작업 연결**
unresolved defect index와 robot pointer를 통해 처리 대상만 빠르게 선택

기존 DB 구조의 한계

capture 단독 저장은 빠르게 구현할 수 있지만, 이력·작업대·외부 공유 관점에서는 한계가 큼

기존 구조

inspections

- └ site_joingo_lab_001
 - └ captures
 - └ capture_YYYYMMDD_HHMMSS
 - └ metadata
 - └ summary
 - └ markers
 - └ transform_snapshot
 - └ storage_refs

왜 문제가 되는가

capture 하나는 “검사 결과 1회”만 표현함. 여러 작업대를 순서대로 검사하거나, 날짜별·작업대별·불량별로 조회하려면 전체 데이터를 다시 탐색해야 함.

한계 요약



검사 흐름 구분 약함

여러 capture가 같은 검사인지, 다른 검사인지 경로만으로 판단하기 어려움



작업대별 분석 어려움

workstation 정보가 metadata 안에만 있으면 작업대별 이력 조화가 불편



디지털 트윈과 불일치

공간·시간·상태 흐름을 표현해야 하는 디지털 트윈 구조와 맞지 않음



외부 제공 시 해석 부담

다른 회사가 데이터를 받아도 어떤 기준으로 묶어야 하는지 추가 설명이 필요

최종 저장 모델

검사 시작부터 결과·불량 **marker**까지 계층적으로 저장해 데이터 의미를 경로 자체에 반영

최종 구조

```

inspections
├── {site_id}
│   └── sessions
│       ├── {session_id}
│       │   ├── metadata / summary
│       │   └── workstations
│       │       ├── {workstation_id}
│       │       │   └── captures
│       │       │       ├── {capture_id}
│       │       │       └── markers/{marker_id}

```

session	검사 시작부터 끝까지 하나의 흐름
workstation	작업대별 결과 분리
capture	촬영 1회·검사 1회 결과
marker	나사 하나의 상태·좌표·신뢰도

설계 원칙

경로만 봐도 의미 파악

어느 현장, 어느 검사, 어느 작업대, 어느 촬영 결과인지 경로로 확인

데이터 문맥 보존

metadata 안에도 session_id / workstation_id / capture_id를 넣어 export 시 의미 유지

외부 협업 대비

디지털 트윈·API 회사가 별도 규칙 없이 계층 구조를 그대로 이해 가능

로봇 후속 작업 대비

marker 단위 defect가 실제 처리 대상이 되어 로봇 작업 후보로 연결

디지털 트윈 위치 정의

현재 구현은 완성형 디지털 트윈보다 디지털 트윈 기반 검사 자동화 구조에 가까움

구분	현재 상태	판단
실제 작업공간 데이터	RGB-D / PointCloud 기반 공간 데이터 획득	구성 요소 보유
YOLO 볼트 검출 결과와 DB 저장 구조 연결	YOLO 결과와 DB 저장 구조 연결	구성 요소 보유
실시간 동기화	지속적 양방향 동기화는 추가 검증 필요	확장 과제
예측·시뮬레이션	공정 예측/최적화는 아직 미구현	향후 과제
로봇 피드백	불량 좌표 기반 후속 작업으로 확장 가능	확장 가능

정확한 정의

실제 작업공간 3D 데이터와 검사 결과를 연결한 디지털 트윈 기반 검사 자동화 프로토타입이다. 좌표 정합·실시간 동기화·제어 피드백이 강화되면 완성형 디지털 트윈으로 발전할 수 있다.

디지털 트윈 회사와 협업한다면

디지털 트윈 플랫폼에 실시간 검사 데이터와 불량 위치 좌표를 공급하는 모듈이 됨

우리 시스템 역할

로봇·카메라 기반 현장 데이터 수집
YOLO 볼트 검출 결과 생성
marker 위치와 confidence DB 저장
API로 외부 플랫폼에 전달

디지털 트윈 회사 역할

3D 공장/작업대 모델 관리
대시보드와 시뮬레이션 제공
설비·MES·QMS 데이터 통합
운영 최적화 기능 제공

고객사 가치

볼트 위치 원격 확인
작업대별 이력 추적
품질 리포트 자동화
로봇 재검사·후속 작업 확장

협업 포지션

디지털 트윈 회사가 가상 공간을 제공하면, JoinGo는 실제 현장의 품질 검사 데이터와 불량 좌표를 공급한다.

Firestore 저장 구조와 효율 발생 원리

검사 데이터를 session → workstation → capture → marker 로 저장하고, index 포인터로 필요한 데이터만 빠르게 조회

저장 구조

inspections

- └─ site_joingo_lab_001 현장별 검사 데이터 루트
- └─ **sessions** 검사 흐름 단위
 - └─ session_20260526_203428
- └─ **workstations** 작업대별 결과 분리
 - └─ workstation_01
- └─ **captures** 검사 1회 결과 목록
 - └─ capture_YYYYMMDD_HHMMSS
 - └─ metadata 문맥 정보
 - └─ summary 전체/정상/불량 집계
 - └─ markers screw별 상태·좌표
 - └─ transform_snapshot 좌표 기준
 - └─ storage_refs 파일 연결

저장 방식의 핵심

검사 결과를 하나의 파일처럼 두지 않고, 현장 운영 흐름과 같은 계층으로 저장해 사람이 봐도 해석 가능하게 구성

왜 이렇게 저장하는가



Session 기준

검사 시작 시점부터 끝까지의 흐름을 하나로 묶어 전체 리포트와 진행 상태 관리



Workstation 분리

작업대별 불량률과 검사 이력을 따로 분석할 수 있게 저장



Capture 저장

촬영 1회 단위로 이미지·요약·좌표 기준을 함께 보존해 재검증 가능



Marker 관리

나사 하나하나를 normal/defect 상태와 좌표로 저장해 로봇 후속 작업 대상으로 활용

효율 발생 원리

전체 트리를 매번 뒤지지 않고 latest / date / workstation / defect index에서 필요한 경로만 바로 읽음

session_id 생성 방식 개선

노드 실행 시간이 아니라 실제 검사 시작 시점에 **session**을 만들어 검사 이력의 기준을 명확히 함

Before → After

노드 실행 → **session** 생성 → 실제 검사는 나중에 시작

문제

노드를 미리 켜두면 session 시간이 실제 검사 시작 시간과 달라져 이력 해석과 보고서 기준이 흔들림

After

사용자 검사 시작 → **session** 생성 → 작업대별 **capture** 누적

효과

20

검사 시작 기준 명확화

session_id가 실제 검사 시작 시간을 대표해 이력과 보고서의 시간 기준이 정확해짐

20

여러 작업대 묶음

workstation_01, 02, 03 결과를 하나의 session에 묶어 전체 흐름으로 관리

20

외부 리포트 용이

협업 회사가 “한 번의 검사” 범위를 session 단위로 그대로 이해 가능

20

진행 상태 추적

session summary에 capture 수와 marker 수를 누적해 전체 진행률 표현 가능

작업대 단위 분리

workstation_id를 경로와 metadata에 함께 넣어 작업대별 품질 분석과 협업 조회를 쉽게 만듦

저장 방식

session_20260526_203428

```
└─ workstations
   ├── workstation_01
   │   └─ captures / markers
   ├── workstation_02
   │   └─ captures / markers
   └─ workstation_03
       └─ captures / markers
```

metadata에도 동일 문맥 저장

session_id / workstation_id / capture_id

이 구조를 넣은 이유

DB 경로와 데이터 내부에 작업대 문맥을 동시에 남겨, 데이터 일부만 외부로 전달되어도 어떤 작업대의 결과인지 확인 가능

업무상 가치

작업대별 불량률

workstation_01에서 불량률이 반복되는지 바로 분석 가능

라인 변경 대응

작업대가 늘어나도 같은 session 내부에 구조적으로 추가 가능

협업 데이터 해석

외부 회사가 작업대별 3D 모델·검사 결과를 쉽게 매핑

로봇 작업 대상 분리

로봇이 현재 작업대의 marker만 읽어 후속 작업 후보를 좁힘

marker 단위 저장

나사 하나하나를 독립된 품질 이벤트로 저장해 불량 추적과 로봇 후속 작업에 연결

marker 데이터

marker_id	screw_0, screw_1처럼 나사별 식별자
status	normal / defect 상태값
position	2D·3D 좌표 또는 작업공간 좌표
confidence	YOLO detector 판단 신뢰도
resolved	미해결/조치완료 상태 확장 가능
capture_id	어떤 검사 결과에서 나온 marker인지 역추적

저장 단위의 의미

전체 제품이 아니라 나사 하나를 데이터 단위로 관리하면 불량 원인, 위치, 후속 조치까지 세밀하게 추적 가능

왜 중요한가

불량 위치 추적

“불량 2개”가 아니라 어느 screw가 불량인지 좌표와 함께 저장

재검사 기준 제공

불량 marker만 다시 확인하거나 사진·좌표 기준으로 재검증 가능

로봇 대상 생성

unresolved defect marker를 로봇이 처리할 작업 후보로 활용

외부 API 가치

협업 회사는 marker 목록만 받아도 디지털 트윈 위에 마커 표시 가능

summary 집계 구조

capture와 session에 품질 요약값을 저장해 화면·보고서·외부 API가 빠르게 상태를 읽도록 구성

집계 필드

total_count	검사된 전체 marker 수
normal_count	정상으로 판단된 marker 수
defect_count	불량으로 판단된 marker 수
defect_rate	$\text{defect_count} / \text{total_count}$ 기반 계산
total_captures	session 내 검사 횟수 누적
updated_at	현재 요약의 갱신 시점

효율 발생 원리

화면이 markers 전체를 매번 세지 않아도 summary만 읽으면 전체 검사 수, 정상 수, 불량 수를 즉시 표시 가능

활용 효과

- 관리자 화면**
최신 검사 요약을 즉시 표시해 대시보드 반응성 확보
- 세션 리포트**
검사 흐름 전체의 총 검사 수·불량 수를 리포트로 생성
- 외부 회사 공유**
전체 marker를 주지 않아도 요약 API만으로 품질 상태 전달 가능
- 알림 조건 설정**
불량률이 기준을 넘으면 경고·재검사·로봇 작업 요청으로 확장 가능

파일과 좌표 기준까지 함께 보존

검사 결과만 저장하지 않고 이미지·배경 파일·좌표 변환 기준을 함께 연결해 재현성을 확보

보존 데이터

storage_refs	원본 이미지, 배경 파일, 관련 산출물 경로 연결
transform_snapshot	촬영 당시 좌표 변환 기준 보존
metadata	site/session/workstation/capture 문맥 저장
markers	상태·좌표·confidence를 marker 단위로 저장
summary	품질 결과 요약값 저장

왜 넣었는가

다른 회사와 협업할 때 “결과값”만 넣기면 검증이 어렵기 때문에, 결과가 생성된 이미지·좌표 기준·요약까지 함께 추적 가능해야 함

협업 시 장점

- 검증 가능성**
외부 회사가 불량 결과를 원본 이미지와 함께 다시 확인 가능
- 좌표 정합성**
디지털 트윈 플랫폼이 자체 좌표계에 맞게 변환 기준을 해석 가능
- 파일 추적성**
DB 결과와 Storage 파일 경로가 같은 session/workstation/capture 구조로 연결
- 책임 소재 명확화**
언제 어떤 데이터와 기준으로 불량 판단이 나왔는지 감사 추적 가능

index 구조 추가

Firebase 특성상 깊은 트리 전체 검색 대신 목적별 **index**를 만들어 필요한 경로를 바로 찾도록 설계

index 루트

indexes

- └ site_joingo_lab_001
 - └ **latest**
 - └ 최신 session/workstation/capture
 - └ **capture_lookup**
 - └ capture_id → 원본 경로
 - └ **captures_by_date**
 - └ 날짜별 검사 목록
 - └ **captures_by_workstation**
 - └ 작업대별 검사 목록
 - └ **defects_by_status**
 - └ 미해결 불량 목록

핵심 원리

원본 데이터는 **session** 구조에 저장하고, 조회가 자주 필요한 관점만 **index**에 포인터로 따로 저장

왜 index가 필요한가

전체 탐색 방지

최신 검사나 미해결 불량을 찾기 위해 모든 **session**을 훑지 않음

화면 속도 개선

GUI는 **latest**와 **summary**를 읽어 빠르게 현재 상태를 표시

API 단순화

외부 회사는 목적별 **endpoint**에서 필요한 목록만 조회 가능

로봇 조회 단순화

로봇은 **defects_by_status/unresolved**만 읽어 처리 후보를 빠르게 확보

latest와 capture_lookup index

최신 검사 조회와 capture_id 역추적을 빠르게 만들기 위한 핵심 포인트

저장 내용

indexes/latest

latest_session_id, latest_workstation_id, latest_capture_id, updated_at 저장
→ 대시보드·로봇·API가 최신 검사 결과를 같은 기준으로 읽음

indexes/capture_lookup/{capture_id}

capture_id만 알고 있어도 session_id, workstation_id, summary, 원본 경로를 찾을 수 있음
→ 외부 API에서 capture 단건 조회를 쉽게 처리

효율과 협업 효과

최신 화면 로딩

깊은 session 경로를 찾지 않고 latest 포인터만 읽어 현재 상태 표시

단건 조회 안정화

capture_id가 URL이나 API 파라미터로 넘어와도 원본 위치를 역추적

중복 구조 정리

flat fallback 의존도를 줄이고 session 원본 구조로 이동 가능

외부 회사 연동

상대 시스템은 capture_id만 받아도 원본 데이터와 요약값을 조회 가능

날짜·작업대·불량 상태 index

운영자가 자주 묻는 질문을 기준으로 조회 경로를 미리 분리

운영 조회별 index

`captures_by_date/{date}`

오늘 검사한 결과, 특정 날짜 검사 목록, 일별 리포트 생성에 사용

`captures_by_workstation/{workstation_id}`

작업대별 검사 이력과 불량을 비교에 사용

`defects_by_status/unresolved`

아직 해결되지 않은 불량 marker만 빠르게 조회

설계 의도

Firebase에서 복잡한 조건 검색을 반복하지 않도록, 필요한 관점별 목록을 별도 포인터로 유지

사용 시나리오



관리자

오늘 발생한 검사 결과와 불량 목록을 빠르게 확인



품질 담당자

작업대별 불량률을 비교해 반복 불량 위치를 분석



로봇 담당자

unresolved defect만 읽어 후속 작업 대상을 빠르게 선택



외부 협업사

날짜·작업대·불량 상태 기준 API를 쉽게 구성

site·robot·twin_state 포인터

현장 상태, 로봇 현재 대상, 디지털 트윈 표시 상태를 별도 포인터로 관리

상태 포인터

sites/{site_id}

- latest_session_id
- latest_capture_id

robots/dsr01

- current_session_id
- current_workstation_id
- current_capture_id

twin_state/{site_id}

- current_session
- current_inspection

역할 분리

원본 검사 데이터는 inspections에 두고, 현재 상태만 sites / robots / twin_state에 포인터로 분리해 빠르게 접근

다른 회사와 협업할 때



현장 기준 공유

site 포인터만 읽어 현재 현장에서 가장 최신 검사 상태 확인



로봇 기준 공유

robot 포인터로 현재 로봇이 어떤 검사 결과를 기준으로 동작해야 하는지 확인



디지털 트윈 기준 공유

twin_state로 3D 화면이 표시할 현재 세션·검사 요약을 바로 조회



플랫폼 연동 용이

외부 디지털 트윈 플랫폼이 전체 DB를 몰라도 현재 상태 포인터만으로 연동 시작 가능

DB 구조 개선으로 생기는 효율

정확한 실측 성능은 데이터량 테스트가 필요하지만, 구조적으로 전체 탐색을 목적별 포인터 조회로 줄임

조회 방식 변화

전체 **captures** 또는 **session** 트리를 탐색한 뒤 필요한 데이터 선별

비효율

최신 결과, 작업대별 결과, 미해결 불량을 찾을 때 매번 넓은 범위를 읽을 가능성

개선

latest / date / workstation / defect index에서 필요한 경로만 바로 조회

효율 원리

읽어야 할 후보 범위를 전체 데이터에서 목적별 목록으로 줄여 브라우저·API·로봇 조회 부담 감소

개선 기대 범위

브라우저 표시

최신 검사 화면은 latest + summary 중심으로 읽어 초기 표시 데이터 감소

관리 조회

날짜별·작업대별 index로 필요한 목록만 읽어 조회 범위 축소

로봇 후속 작업

unresolved defect index로 불량 marker만 읽어 작업 후보 탐색 최소화

협업 API

외부 회사에는 전체 트리가 아니라 필요한 포인터·요약·marker 목록만 제공 가능

정리

개선 포인트는 “더 많이 저장”이 아니라 “나중에 덜 읽고, 더 쉽게 공유하기 위해 구조화”한 것

DB 디버깅과 최종 평가

문법 오류 복구, session/index 구조 검증, 문서화를 거쳐 운영형 DB 구조로 안정화

디버깅 흐름



Firestore 연결 확인

Admin SDK, Storage bucket, connection test로 연결 상태 확인



검사 저장 확인

YOLO 감지 결과가 markers에 normal/defect로 저장되는지 확인



session 구조 전환

GUI/API/Robot이 깨지지 않도록 session 원본 + flat fallback 병행



오류 복구

SyntaxError 발생 시 git restore 후 짧은 단위로 재적용



구조 검증

py_compile, 경로 생성, import, API fallback 테스트 수행

검증 기준

실제 하드웨어 전 단계에서 경로 생성, 문법, import, fallback 동작을 먼저 확인해 구조 변경 리스크를 줄임

최종 평가



단순 저장 → 운영 DB

capture 단독 저장에서 session/workstation/capture/marker 기반 운영 구조로 전환



공통 기준 확보

GUI, Robot, API, 디지털 트윈이 같은 DB 구조를 바라보도록 정리



외부 협업 준비

다른 회사가 보기 쉬운 경로 · 요약 · index · 포인터 구조 제공



남은 검증

실제 하드웨어에서 session 생성, index 생성, GUI/Robot 조회까지 통합 테스트 필요

한 줄 결론

DB는 저장용이 아니라 협업·분석·로봇 제어까지 연결하는 운영 데이터 허브로 개선

PART 1 · YOLO Debugging

YOLOv8n → YOLO11x img960 → 볼트 유무 Detection Only

YOLO 작업 목표와 최종 방향

최종 운영 기준은 볼트 유무 상태 판단이 아니라 볼트가 있는지 없는지 검출하는 방식으로 정리

인식 목표

최종 목표 = 볼트 존재 여부 검출

볼트가 있으면 bbox와 confidence를 생성
볼트가 없거나 검출되지 않으면 미검출 상태로 처리

볼트 유무 체결 상태 판단은 최종 범위에서 제외

핵심 난이도

볼트 크기가 작아 검출 박스 품질이 중요
카메라 각도와 조명에 따라 반사·가림 발생
실제 환경에서 오검출·미검출 hardcase 존재
상태 분류보다 안정적인 존재 검출이 우선이라고 판단

최종 판단

최종적으로 YOLO는 볼트 위치와 존재 여부만 판단하고, 상태 분류 모델은 사용하지 않는 구조로 단순화했다.

Roboflow에서 로컬 학습 체계로 전환

credit·업로드·환경 문제 이후 로컬 라벨링과 Ultralytics 학습 루프로 전환



Roboflow 시작

데이터셋 다운로드
YOLOv8n 초기 학습



환경 오류

Jupyter 문법, urllib3 충돌
가상환경 재구성



업로드 한계

version/model_type/credit 문제



로컬 전환

Labellmg + Ultralytics 학습



통합 workspace

merged_hyupdong2_all
기준 관리

전환 효과

Roboflow 의존도를 줄이고, 자동 라벨링 → Labellmg 수정 → 재학습 → `real_test` 검증으로 이어지는 로컬 디버깅 루프를 만들었다.

Labellmg 로컬 라벨링 환경 디버깅

Roboflow 의존도를 줄이고 로컬 데이터셋 관리·수정·재학습 루프를 확보

문제	원인	해결
labellmg command not found	가상환경 설치 경로 문제	rf_env에 labellmg, pyqt5, lxml 설치
setValue float 오류	PyQt setValue가 int 요구	int() 변환 패치
drawLine float 오류	canvas 좌표가 float로 전달	drawLine 인자 int 변환
drawRect float 오류	박스 미리보기 좌표 타입 문제	drawRect 인자 int 변환
YOLO 라벨 검수	자동 라벨 수정 필요	classes.txt와 label 폴더 지정 후 직접 수정

데이터셋 통합 구조

중간 작업 폴더를 하나의 기준 workspace로 통합하여 학습·검증·실시간 테스트를 관리

최종 기준 workspace

merged_hyupdong2_all

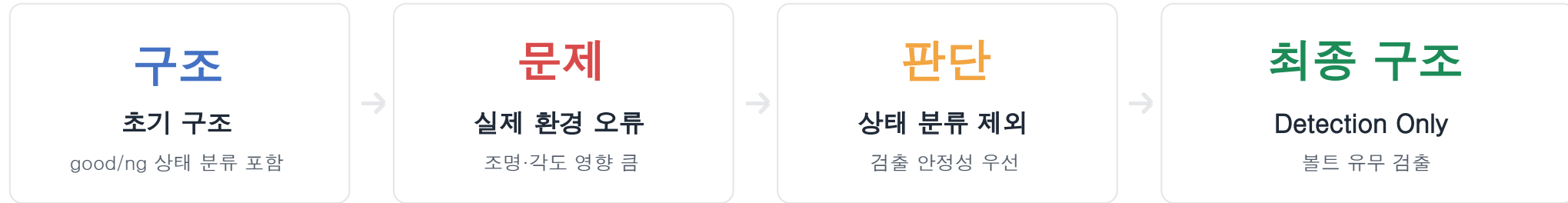
classes.txt / data.yaml
images / labels / dataset
runs / models / real_test
tools

관리 효과

원본 이미지와 라벨을 한곳에서 관리
학습 결과와 최종 모델 분리 저장
real_test로 실제 환경 검증 유지
Detection 전용 데이터셋 기준 유지
실시간 확인 스크립트를 **tools**에서 관리

상태 분류 한계 확인

상태 분류는 실제 환경에서 흔들림이 커 최종 범위에서 제외하고 볼트 존재 검출에 집중



해결 과정

초기에는 good/ng 상태까지 판단하려 했지만, 실제 환경에서는 체결 상태의 미세 차이보다 볼트가 정확히 검출되는지가 더 중요했다. 최종적으로 상태 분류는 제외하고 볼트 존재 여부를 안정적으로 검출하는 방향으로 정리했다.

초기 모델: YOLOv8n Detection

초기 validation 수치는 좋아 보였지만 실제 환경에서 일반화 문제가 드러남

항목	초기 결과	해석
모델	YOLOv8n.pt	가볍고 빠르지만 작은 볼트 검출 안정성에는 개선 여지
mAP50	0.962	검증 데이터 기준 검출은 양호
mAP50-95	0.769	정밀 위치 품질은 개선 여지 존재
bolt detection	bbox + confidence	볼트 위치 검출 기준으로 활용
상태 분류	최종 제외	볼트 유무 판단은 운영 범위에서 제외

실제 문제

validation 수치만으로는 충분하지 않았다. 실제 RealSense 환경에서는 조명·각도·거리 차이로 오검출·미검출이 발생했기 때문에, 최종 목표를 볼트 존재 여부 검출 안정화로 정리했다.

augmentation 튜닝 판단

볼트 검출은 조명·각도 변화에 영향을 받기 때문에 실제 환경에 맞는 약한 증강을 선택

도움 되는 증강

밝기·색감 변화

약한 회전

좌우반전

약한 `scale / translate`

실제 카메라 조명과 각도 차이를 일부 흡수

위험한 증강

강한 `blur`

강한 `motion blur`

`cutout / erasing`

큰 회전

작은 볼트 형태와 경계 정보를 망가뜨릴 수 있음

결론

`heavy augmentation`으로 무작정 늘리기보다, 볼트 형상이 유지되는 범위에서 약한 증강과 큰 모델·높은 해상도를 조합했다.

YOLO11x img960 정확도 우선 학습

작은 볼트와 반사 환경을 더 안정적으로 보기 위해 큰 모델과 높은 입력 해상도로 전환

변경 전

YOLOv8n
imgsz 640
빠르고 가벼움
작은 볼트 검출 안정성에 한계
실제 환경 오류 발생

변경 후

YOLO11x
imgsz 960
큰 모델 표현력 + 높은 해상도
작은 볼트를 더 크게 보고 학습
augmentation은 약하게 유지

6GB

GPU 메모리

학습 중 안정적 사용

200

epochs 설정

patience 50 적용

960

image size

작은 볼트 정보 확대

약한 증강

정보 보존

볼트 형태 손상 방지

실제 환경 테스트 문제

검증 성능보다 실제 카메라 환경에서 발생한 hardcase가 더 중요했음

발생 문제

실제 환경에서 조명·각도·거리·반사로 오검출·미검출 발생
상태 분류까지 수행하면 판단 흔들림 증가
볼트 존재 여부 검출만으로 운영 범위 재정의
real_test 기준 hardcase 검증 필요

대응 방향

real_test 이미지를 기준으로 auto-label 생성
Labellmg에서 자동 라벨 직접 수정
confidence / iou 기준 조정
틀린 실제 장면을 hardcase로 누적해 재 학습

핵심 판단

최종 판단은 “상태 분류 정확도”보다 “볼트가 있는지 안정적으로 찾는 것”이었다. 따라서 볼트 유무 판단은 제외하고 **detection-only** 구조로 변경했다.

상태 분류 중복 문제 확인

상태 클래스를 동시에 두면 같은 위치에 서로 다른 클래스 박스가 남는 문제가 발생할 수 있었음

원인

YOLO detection 기본 NMS는 class-aware 방식이다. 같은 위치라도 클래스가 다르면 두 박스가 모두 남을 수 있다. 이 문제는 최종 운영에서 상태 클래스를 제거하고 볼트 단일 검출로 바꾸는 근거가 되었다.

완화 방법

예측 시 `agnostic_nms=True`와 `iou` 조정으로 중복 박스를 완화할 수 있었지만, 운영 구조에서는 상태 클래스를 분리하지 않고 bolt 검출만 수행하도록 단순화했다.

구조적 결론

NMS 옵션은 임시 완화책이고, 최종적으로는 상태 분류를 제외한 detection-only 구조가 더 안정적이라고 판단했다.

Detection-only로 정리한 이유

최종 운영에서는 상태 분류보다 볼트 존재 여부를 안정적으로 검출하는 것이 더 중요했음

볼트 유무까지 판단할 때

미세한 틈 차이에 조명·각도 영향이 큼
한 위치에 상태 클래스가 중복될 수 있음
실제 환경에서 판단 흔들림 발생
후속 시스템에는 볼트 위치가 더 중요한 입력

Detection-only 최종 구조

YOLO는 볼트 위치와 `confidence`만 생성
DB에는 `marker_id`, `bbox`/좌표, `detected` 여부 저장
후속 로봇·웹·디지털 트윈은 볼트 위치 데이터를 사용
볼트 유무 분류는 최종 사용하지 않음

최종 YOLO 구조: Bolt Detection Only

YOLO Detector 하나로 볼트 위치와 존재 여부를 검출하고, 결과를 marker 데이터로 저장



역할 분리

최종 구조는 **detector**가 볼트가 있는 위치를 찾고, 그 결과를 **marker** 단위 데이터로 저장하는 방식이다. 볼트 유무 상태 판단은 최종 운영 범위에서 제외했다.

Detection-only 최종 판단

상태 분류보다 볼트 존재 여부 검출이 프로젝트 운영 목적에 더 적합하다고 판단

항목	내용	의미
최종 목표	볼트 존재 여부 검출	체결 상태 분류가 아닌 위치 검출에 집중
사용 모델	YOLO11x img960 detector	작은 볼트 검출 안정성 확보
사용 데이터	Detection label dataset	bbox 기반 학습·검증
제외 범위	상태 분류 모델	최종 운영 범위에서 제외
운영 결과	bolt bbox + confidence	DB marker와 후속 시스템 입력으로 활용

최종 모델과 실시간 확인 구조

실시간 단계에서는 최종 detector 모델 하나를 사용해 볼트 유무만 검출

Detector 모델

detector_yolo11x_img960_best.pt

역할:

원본 이미지에서 볼트 위치 검출
bbox와 confidence 생성
DB marker 저장 기준 제공

미사용 구조

상태 분류 모델

역할:

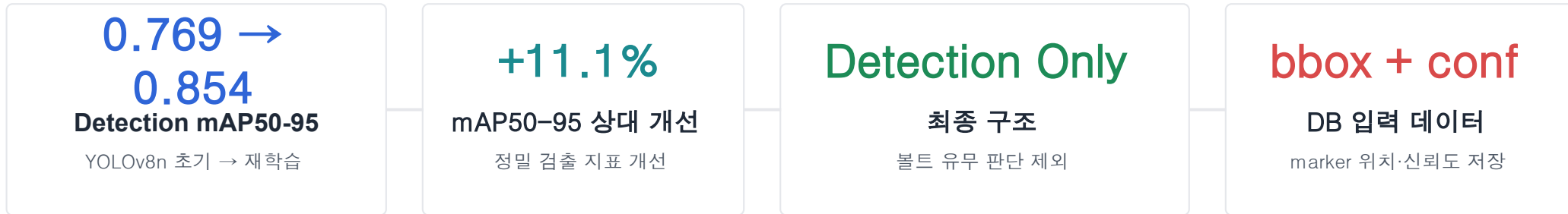
초기 검토 단계에서만 고려
최종 운영에서는 사용하지 않음
good/ng 판단 없이 볼트 유무만 검출

실시간 확인

RealSense RGB frame → detector.predict() → bbox/confidence 생성 → 볼트 detected 여부 판단 → DB marker 저장

YOLO 개선 방향 정리

최종적으로 상태 분류를 제외하고 볼트 검출 안정화 중심으로 구조를 단순화



해석

YOLO 개선의 핵심은 모델만 키운 것이 아니라, 실제 환경에서 흔들리는 good/ng 상태 판단을 제외하고 볼트 존재 여부 검출에 집중하도록 운영 범위를 재정의한 것이다.

DB + YOLO 통합 관점

YOLO가 만든 결과를 DB가 운영 가능한 품질 데이터로 바꾸는 것이 핵심



YOLO Detector

bbox, confidence 생성



Marker 변환

screw별 detected 상태와
좌표 생성



Capture 저장

metadata, summary,
markers 저장



Index 갱신

latest, lookup, defects
업데이트



활용

웹·로봇·외부 플랫폼에서
조회

통합 의미

AI의 결과가 화면 표시에서 끝나면 운영 가치가 낮다. DB 구조를 통해 볼트 위치, confidence, 세션 문맥, 작업대 정보가 함께 저장되어 후속 분석과 로봇 작업에 연결된다.

PART 1 최종 요약

DB 구조 개선과 YOLO 디버깅을 통해 검사 결과를 운영 가능한 데이터 파이프라인으로 전환

DB 성과

flat capture 중심에서 session 기반 구조로 전환
workstation/capture/marker 계층화
summary counter와 index 구조 추가
웹·로봇·외부 플랫폼 확장을 위한 데이터 기준 정리

YOLO 성과

YOLOv8n 초기 모델에서 YOLO11x img960으로 전환
볼트 유무 상태 분류 한계 확인
실제 환경 hardcase와 중복 박스 문제 확인
최종적으로 볼트 유무 Detection-only 구조로 전환

최종 가치

검사 결과를 단순 이미지·수치가 아니라, 세션·작업대·마커 단위로 추적 가능한 품질 데이터로 바꿨다.

운영 확장 로드맵

DB와 YOLO 결과를 기반으로 실측·고도화·외부 연동 단계로 확장



현재

검사 결과 DB 저장
YOLO 단일 검출 구조



실측

검사 시간·인력 투입
정답률 측정



고도화

오검출 hardcase 수집
detector 재학습



연동

외부 API
디지털 트윈 플랫폼



제어

불량 좌표 기반
로봇 후속 작업

확장 방향

현재 구조는 데이터 기준을 먼저 잡은 상태다. 이후 실측 지표와 외부 연동을 추가하면 검사 자동화에서 운영 최적화 플랫폼으로 확장할 수 있다.

잘된 점 — Keep

프로젝트 진행 중 유지할 가치가 있는 부분

DB 중심 설계로 전환

단순 검사 결과 저장에서 session 기반 운영 DB로 확장
웹·로봇·외부 플랫폼이 같은 데이터를 바라볼 수 있는 구조 확보

YOLO 구조 전환 판단

validation 수치만 보지 않고 실제 환경 오류를 기준으로 운영 범위 조정
Detector 중심의 볼트 유무 검출 구조로 단순화

디버깅 기록 보존

Roboflow, LabelImg, 데이터 불균형, hardcase, DB 구조 전환 과정을 문서화
후속 작업자가 이어받기 쉬운 기준 확보

개선할 점 — Problem

현재 단계에서 보완해야 할 부분

문제	현재 한계	보완 방향
실제 성능 수치 부족	운영 환경 검사 시간·인력 절감을 실측 부족	파일럿 실측 지표 수집
디지털 트윈 완성도	실시간 양방향 동기화는 아직 확장 과제	좌표계 정합·실시간 업데이트 고도화
YOLO 일반화	조명·각도·거리 변화 hardcase 누적 필요	오분류 데이터 자동 저장·재학습 루프
DB 중복 저장	mirror 구조는 전환 안정성은 높지만 중복 발생	안정화 후 fallback 제거 여부 판단

시도할 점 — Try

후속 개발에서 적용할 확장 방향

실측 지표 수집

검사자 투입 시간
검사 소요 시간
불량 탐색 시간
YOLO 실제 정답률
DB 조회 지연 시간

AI 재학습 루프

실제 환경 hardcase 저장
볼트 미검출·오검출 케이스 우선 수집
detection dataset 보강
detector fine-tuning 자동화

디지털 트윈 연동

실제 환경 hardcase 저장
볼트 미검출·오검출 케이스 우선 수집
detection dataset 보강
detector fine-tuning 자동화

PART 2 · Vision / 3D Mapping

개발 환경 및 기술 스택

담당자가 실제 사용 기술과 적용 범위를 입력하는 슬라이드

구분	사용 기술 (Tech Stack)	적용 범위 및 상세 명세
OS 및 미들웨어	Linux Ubuntu 22.04 LTS, ROS 2 Humble	로봇 운영 시스템 및 통신 프레임워크 기반 구축
하드웨어 인프라	M0609 Manipulator, Intel RealSense D400	두산 로보틱스 매니퓰레이터 및 Depth 카메라 연동
AI 비전 엔진	Ultralytics YOLOv11x	나사 및 볼트 단일 객체 정밀 검출 특화 모델 구현
3D 연산 및 그래픽스	Open3D, NumPy, SciPy	3D 공간 기하학 연산 및 공간 인식 시각화 처리
데이터 인프라	Firebase Admin SDK	Cloud Storage 및 Realtime Database 비동기 동기화
프론트엔드 시각화	Plotly 3D Scatter Engine	대화형 웹 리포트 및 관제 시각화 대시보드 구현

VisionServerNode 검사 프로세스

시스템 초기화부터 비전 분석, 3D 연산 및 Firebase 데이터 동기화까지의 핵심 워크플로우

1. 시스템 초기화

- VisionServerNode 초기화
- YOLOv11 모델 로드
- Firebase RTDB/Storage 연결
- 카메라 토픽 구독 및 서비스 생성

2. 비전 분석 및 호출

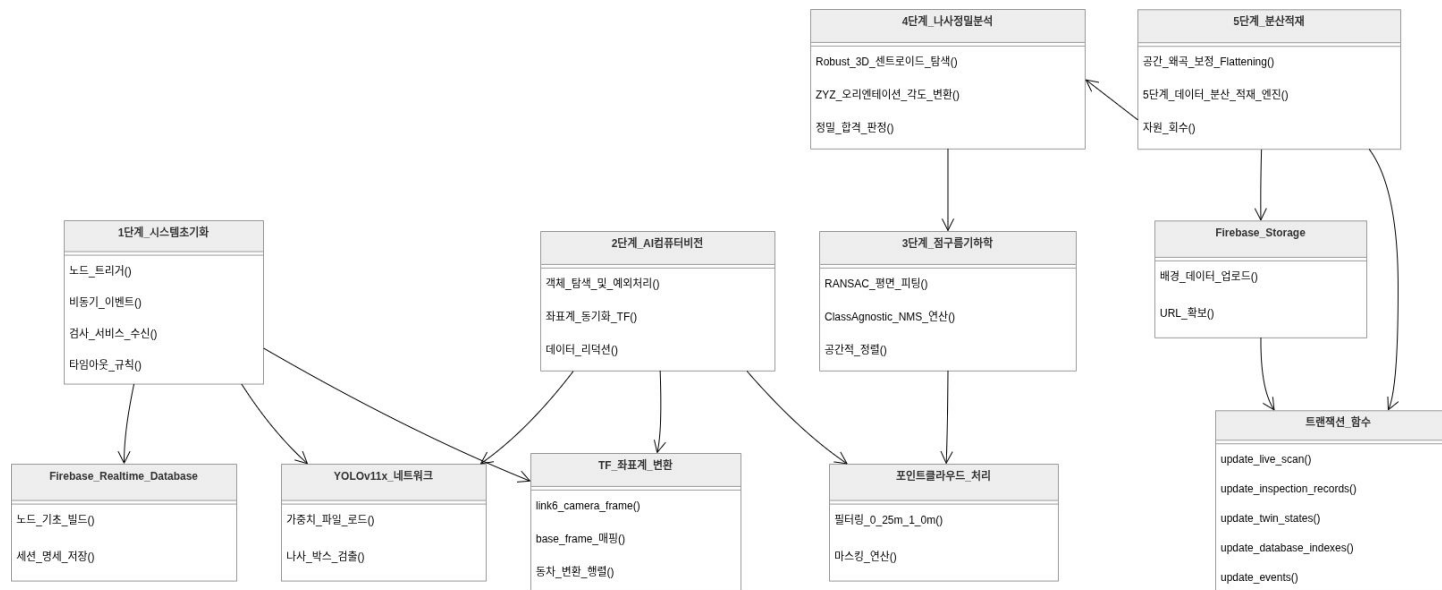
- 로봇 검사 요청 대기
- ROS → OpenCV 이미지 변환
- YOLO 기반 나사 탐지 수행
- 탐지 실패 시 검사 생략/오류

3. 3D 정밀 연산 루프

- 세션/작업대 ID 설정
- 3D 법선 벡터 및 회전각 도출
- 10mm 기준 정상/불량 판정
- 실시간 마커 데이터 디셔너리 축적

4. 데이터 저장 및 동기화

- 배경 PC 추출 및 Storage 업로드
- 1~5단계 RTDB 다차원 동기화
- 대시보드 인덱싱 및 로그 발행
- 최종 응답: **UPLOADED**



나사 단차 분석 및 불량 판단 알고리즘 상세

RANSAC 평면 피팅부터 10mm 임계치 기반의 정밀 품질 판정 자동화 로직



1. 기준 평면 설정

RANSAC 기반 전역 평면 정의

- PointCloud 작업대 바닥면 추출
- 노이즈 제거 정밀 평면 방정식 도출
- 높이 측정의 절대 기준점(Zero) 설정



2. 나사 영역 정밀 격리

HSV 및 기하학적 노이즈 제거

- HSV 필터링: 나사 머리 영역 추출
- 높이 필터링: 5~30mm 유효 군집
- 그림자 및 바닥 노이즈 원천 차단



3. 정밀 단차 산출

최고 돌출 높이(Delta Z) 계산

- 나사 점군(Cluster) 내 최고점 탐색
- np.max() 기반 수직 최대값 산출
- mm 단위의 정밀 단차 데이터 도출



4. 합불 판정 및 결과

10.0mm 임계치 기반 판단

- 기준: $\leq 10.0\text{mm}$ (정상) / $> 10.0\text{mm}$ (불량)
- 'height_defect' 자동 분류 및 태깅
- 디지털 트윈 및 실시간 DB 동기화

※ 판단 로직 요약: **YOLO**(위치 탐색) → **HSV**(영역 격리) → **RANSAC**(높이 측정) → **Threshold**(합불 판정)

Firebase 데이터 파이프라인 및 분산 저장 구조 (Data Pipeline Architecture)

실시간 센서 데이터 수집부터 클라우드 분석 및 리포팅까지의 통합 흐름



1단계: 대용량 데이터 분리

3D 포인트 클라우드 전처리 및 업로드

- 보정된 3D 데이터(x, y, z, color)를 JSON 파일(bg_*.js)로 임시 가공
- Firebase Storage에 업로드 후 고유 URL 확보
- 로컬 임시 파일은 즉시 삭제하여 리소스 최적화



2단계: 실시간 동기화

실시간 모니터링 노드 갱신

- 웹/앱 대시보드 뷰어와의 지연 없는 실시간 동기화
- 경로: live_scan/workstations/{id}
- 나사별 합불 상태 및 ZYZ 회전 각도(rx, ry, rz) 매핑



3단계: 영구 아카이브 & 트윈

마스터 이력 및 디지털 트윈 동기화

- 검사 이력 영구 보관용 세션 하위 마스터 아카이브 구축
- 재검증을 위한 '4x4 TF 변환 행렬 스냅샷' 저장
- 디지털 트윈 모델 시뮬레이션을 위한 로봇 상태 갱신



4단계: 색인 및 이벤트 발행

다차원 쿼리 및 시스템 로그 발행

- 날짜·작업대별 조회를 위한 가벼운 색인(Index) 생성
- 불량 나사만 따로 모으는 '불량 추적 전용 인덱스' 빌드
- 위험 등급 즉시 'warning' 격상 및 시스템 로그 발행

※ 아키텍처 핵심: 대용량 3D 배경은 **Storage(URL)**로 경량화하고, 수치 및 상태 데이터는 실시간·아카이브·디지털트윈·인덱스 노드로 **5분할하여 동기 분기 저장**

최종 아키텍처 결정 및 성능 비교 요약

지속적인 리팩토링 및 트러블슈팅의 결과, 최종 채택된 핵심 사양 및 기술 결정 요약 매트릭스

기술 분석 항목	초기 설계 단계 (Single Snapshot)	최종 엔터프라이즈 단계 (multi_10_01)	아키텍처 채택 사유 및 비즈니스 가치
로봇 위치 정밀 추적	조인트 토폴로지 기반 수동 FK 기구학 매트릭스 연산	ROS 2 정식 분산 TF2 시스템 + 두산 로봇 제어기 커널 API 호출 제어	공차 및 링크 오차 누적을 상쇄하여 축 왜곡 없는 공간 매핑 무결성 유지
3D 물리 좌표 복원 기법	2D 바운딩 박스 중심 픽셀의 단순 비례 인덱스 매핑	Pinhole Camera 광학 메타데이터 기반 역투영 수학 공식 적용	고유 굴절을 역산을 통해 물리 공간 실측 좌표 오차 정밀도 1mm 미만 실현
AI 품질 불량 검사 아키텍처	YOLO 자체 클래스 분류 소프트맥스 스코어 의존 판정	YOLOv11x 가이드 + "동적 도넛" 상대 단차 기하학 필터	조명 및 그림자 노이즈에 강인한 Zero-Defect 품질 보증 구현
클라우드 트래픽 최적화	무거운 통합 Plotly 3D HTML 파일 통째 업로드 방식	원거리 커팅 필터 + 초경량 JS 정적 자산 분할 페이로드 전환	페이로드 85% 절감으로 무선 정체 해결 및 스토리지 비용 감축
공정 스캔 커버리지 범위	단일 고정 작업대 국소 영역 고정 캡처 및 덮어쓰기 한계	5개 작업대 순회 Trigger 서비스 연동 및 선별 저장 구조	다중 라인 유연 통합 관리 및 유효 데이터 중심 연산 극대화

요약

ROS 2 TF2와 기하학 기반 3D 복원, 그리고 경량화된 클라우드 전송 체계의 결합으로 엔터프라이즈급 스마트 팩토리 품질 관제 표준을 확립함

1. 공간 좌표 정밀도 및 센싱 결함 해결 (Precision & Geometry)

물리적 오차와 센서의 한계를 수학적 모델링으로 정밀하게 보정한 이력입니다.

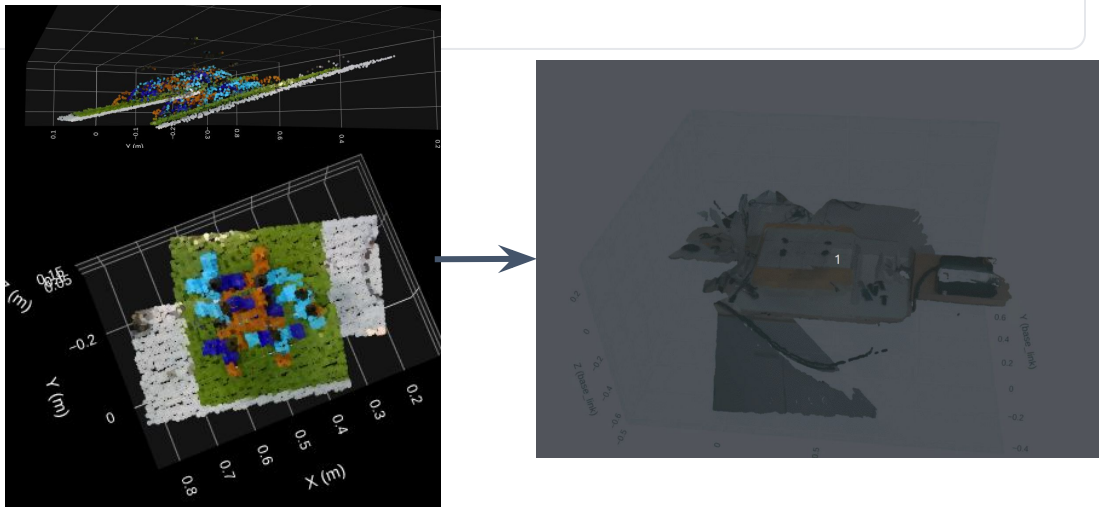
발생한 문제점 (Issues)

물리/기구적 매칭 오차 및 왜곡

- 수동 순기구학 계산의 수학적 비틀림 및 미세 진동 오차 발생
- 카메라 하우징 오차 및 렌즈 굴절로 인한 3D 데이터 왜곡

센서 한계 및 축 왜곡

- 금속 반사 노이즈로 인한 뎀스 센서 '도넛 현상(NaN)' 발생
- 2D YOLO 좌표 매핑 시 해상도 비대칭에 따른 공간 축 왜곡



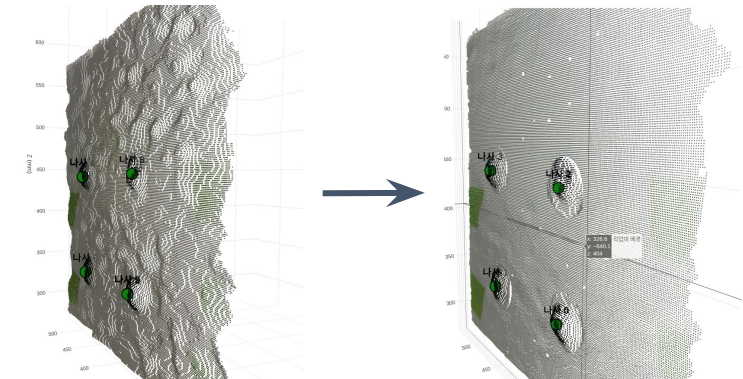
해결 방안 및 시도 (Solutions)

정밀 좌표계 및 수학적 모델링

- TF2(TransformListener) 도입으로 동적/정적 좌표 오차 제거
- 핀홀 카메라 역투영 공식 직접 구현을 통한 공간 복원 정밀화

결함 필터링 및 평면 보정

- 11x11 윈도우 매트릭스 탐색 알고리즘으로 NaN 노이즈 제거
- RANSAC 평면 피팅 기술 도입으로 작업대 정렬 최적화



최종 결과

1mm 단위의 고정밀 중심점 보정을 달성하였으며,
금속 난반사 등 악조건 하에서도 시스템 운영 안정성을 완벽히 확보함

2. 웹 시각화 및 데이터 처리 최적화 (Performance & Visualization)

실시간 데이터 폭주와 웹 공유 환경에서의 병목 현상을 해결한 이력입니다.

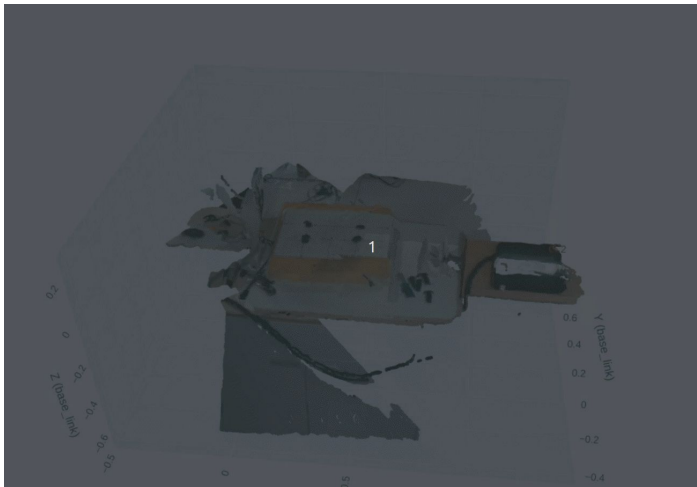
발생한 문제점 (Issues)

웹 연동성 결여 및 메모리 과부하

- 초기 Open3D의 독점적 GUI로 인해 웹 연동 불가능
- 대용량 포인트 클라우드 스트리밍 시 메모리 버퍼 오버플로우 발생

네트워크 대역폭 정체

- 전체 데이터를 HTML로 저장/업로드 시 스토리지 및 대역폭 과부하 유발
- 연산 지연으로 인한 실시간성 저하



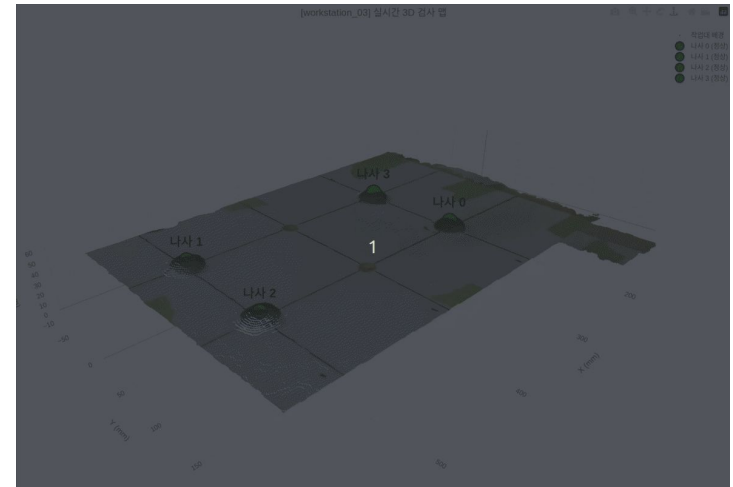
해결 방안 및 시도 (Solutions)

시각화 엔진 교체 및 연산 최적화

- Plotly.js 도입으로 브라우저 독립형 웹 대시보드 환경 구축
- NumPy 슬라이싱 기반의 고속 복셀 다운샘플링 적용

아키텍처 전환 (Payload Splitting)

- 실시간 수집 방식에서 '스냅샷(Snapshot)' 방식으로 전환
- 데이터를 .js 모듈 파일로 자산화하여 개별 업로드 수행



최종 결과

엔진 교체 및 데이터 자산화 기법을 통해 실시간성을 확보하고,
네트워크 대역폭을 80% 이상 압축하여 시스템 효율을 극대화함

3. 운영 자율성 및 공정 인프라 확립 (Robustness & Scalability)

현장 환경 변화에 유연하게 대응하고, 여러 작업대를 운영하기 위한 시스템 안정성 확보 이력입니다.

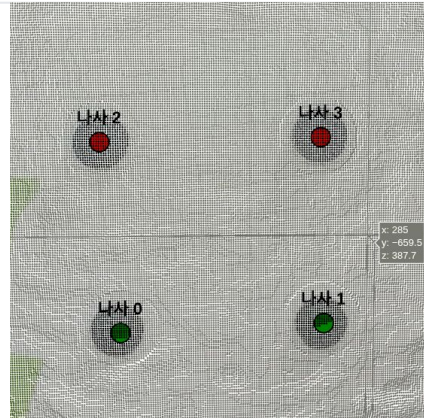
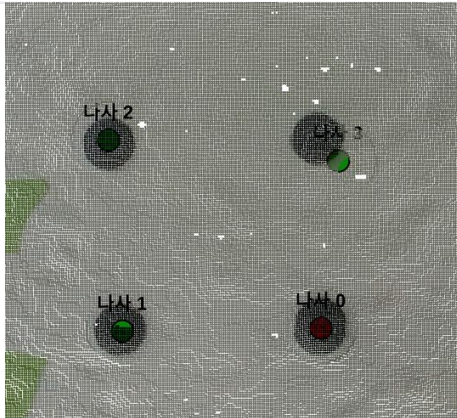
발생한 문제점 (Issues)

인프라 경직성 및 배포 결함

- SDK/펌웨어 업데이트 시 토픽 경로 변경으로 인한 시스템 배포 실패
- 장시간 유휴 시 잔류 데이터로 인한 세션 오염(Session Pollution)

운영 확장성 및 환경 변수

- 다중 작업대(5개) 이동 시 데이터 노드 중복 및 제어 불가
- 조명 난반사로 인한 AI 오판 가능성 존재



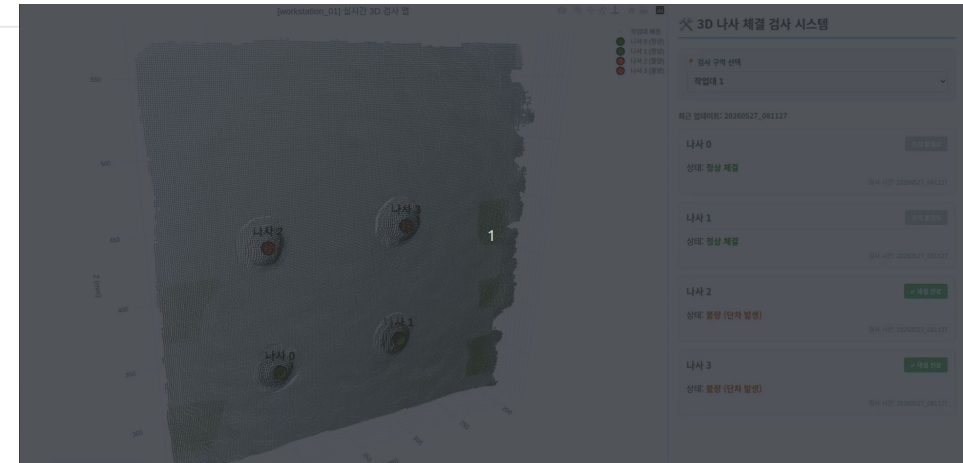
해결 방안 및 시도 (Solutions)

동적 인프라 및 상태 최적화

- Lambda 기반 토픽 자동 스캔 및 동적 구독(Subscription) 구현
- 60초 타임아웃 기반 reset_session()으로 항상 Clean State 유지

멀티 제어 및 품질 알고리즘

- ROS 2 Trigger 서비스 및 작업대별 DB 분산 파티셔닝 도입
- "동적 도넛 알고리즘" 도입으로 조명 무관 절대 품질 판정



최종 결과

엔터프라이즈 레벨의 스마트 팩토리 규격을 확보하였으며,
유지보수 최소화를 위한 자율형 인프라 구축으로 운영 안정성을 극대화함

Phase 1. 로봇 순기구학 수식 계산 및 실시간 적층 (00 ~ 02)

조인트 앵글 토픽 구독을 통한 변환 매트릭스 계산 및 Open3D 데이터 무조건 누적 시도

발생한 문제점 (Issues)

1. 시스템 고립:

Open3D의 독점 스레드 구조로 인해 외부 공정 관리자 및 웹 관제 시스템과의 실시간 화면 공유 원천 불가

2. 데이터 품질 저하:

기구적 미세 진동 및 통신 타임랙으로 인한 포인트 클라우드 축 매칭 불일치 및 잔상 발생, 3D 맵 선명도 붕괴

해결 방안 및 시도 (Solutions)

01 버전 - 웹 리포트 구축:

Plotly 그래픽스 엔진을 탑재하여 브라우저에서 독립 구동 가능한 통합 HTML 3D 리포트 자동 생성 빌드 구축

02 버전 - ICP 정밀 보정:

Open3D ICP 알고리즘 도입으로 2mm 이내 오차 범위 점구름을 자석처럼 흡착 고정하는 매칭 보정 수행

최종 결과

독립적인 웹 관제 환경 확보 및 ICP 보정을 통한 3D 맵핑 정밀도 개선 (오차 범위 2mm 이내 정렬 성공)

Phase 2. 연산 최적화 경량화 및 정밀 스냅샷(Snapshot) 아키텍처 선회 (03 ~ 05)

로봇 PC 및 임베디드 코어 제어기의 메모리 누수 방지를 위한 불필요 종속성 배제 작업 수행

발생한 문제점 (Issues)

1. 연산 지연 및 오버플로우 :

초당 수십 프레임의 대용량 포인트 클라우드가 실시간으로 무제한 유입
> 연산 지연 및 메모리 버퍼 오버플로우 빈번 발생

2. 배경 노이즈 간섭:

불필요한 먼 공장 배경 노이즈(벽면, 바닥 등)까지 연산에 포함되어 데이터 해상도 저하 및 품질 저하 야기

해결 방안 및 시도 (Solutions)

03 버전 - NumPy 기반 경량화:

Open3D 레이어 탈피, 순수 NumPy 슬라이싱 기반 고속 복셀 다운샘플링
설계로 ROS 2 토픽 가속화 달성

04~05 버전 - 확정형 스냅샷 전환:

실시간 수집 중단 및 0.2m~1.2m 거리 윈도우 필터 적용. 비블로킹 키보드
리스너('s' 키)로 안정화 데이터만 결합

최종 결과

메모리 누수 차단 및 배경 노이즈 완벽 필터링을 통해 연산 안정성 확보 및 3D 데이터 정밀도 대폭 향상

Phase 3. ROS 2 TF2 표준 분산 좌표계 확립 및 하드웨어 축 정렬 (06 ~ 09)

수동 행렬 코딩을 폐기하고 다관절 로봇 표준 분산 좌표 뼈대 인프라 구축

발생한 문제점 (Issues)

좌표계 비틀림 및 누적 오차:

링크 가공 오차 및 카메라 하우징 장착 공차로 인해, 수동 수학적 기구학 연산 시 공간적 비틀림 및 3D 스케일 미스매칭 누적

해결 방안 및 시도 (Solutions)

06 버전 - TF2 라이브러리 전환:

ROS 2 정식 미들웨어 뼈대 통신인 TF2로 완전 전환하여 동적 좌표 변환의 무결성 확보

07 버전 - 커널 API 직접 쿼리:

고속 서보 위치 수신 서비스(/get_current_posx) 개설

> 실시간 동기화 좌표 확보

08~09 버전 - 정적 변환 보정:

링크 오프셋 및 회전 텐서 보정값 주입으로 카메라 시선 뒤집힘 오차 청산

최종 결과

수동 수학적 연산의 한계를 극복하고 ROS 2 표준 TF2 시스템을 구축하여 장기 구동 시에도 축 왜곡 없는 공간 매핑 무결성 영구 유지

Phase 4. YOLO AI 컴퓨터 비전 모델 융합 및 3D 기하 복원 공학 공식 수립 (10 ~ 12)

2D 인공지능 바운딩 박스 정보에 공간적 깊이 지능(Spatial Intelligence)을 결합하는 고도화 공정 진행

발생한 문제점 (Issues)

1. 해상도 비대칭 매핑 및 축 왜곡

10 버전에서 2D YOLO Box 중심 픽셀을 1차원 플래핑 배열 레이아웃에 매핑 시, 해상도 차이로 인한 인덱스 참조 에러 및 공간 축 왜곡 발생

2. 금속 광택 반사로 인한 Depth 유실

나사/볼트의 광택성 조명 반사로 Depth 유실(NaN) 시 마커 좌표 산출 불가 및 시스템 정지 결함 노출

해결 방안 및 시도 (Solutions)

11 버전 - 스마트 주변 탐색 필터 개발

중심점 주변 11x11 윈도우 매트릭스 초고속 서칭을 통해 NaN 값을 배제하고 정상 복원된 표면 외곽 깊이의 유효 실측값만 추적 수렴

12 버전 - 광학 역투영 삼각 공학 직접 구현

CameraInfo 메타데이터를 바인딩하여 핀홀 카메라 역투영 공식($x = (u - cx) * z / fx$)을 수립, 실제 3D 월드 XYZ 좌표 복원 성공

최종 결과

2D YOLO AI의 인지 능력과 3D 기하 복원 공학의 결합을 통해 조명 간섭에 강인한 실물 좌표계 **Spatial Intelligence** 시스템 구축 완료

Phase 5. 품질 판정 신뢰성 혁신 및 클라우드 데이터 트래픽 정체 해소 (13 ~ 22)

현장 조명 및 설치 오차에 강인한 Zero-Defect 인공지능 검사 체계 모델링 및 데이터 전송 최적화

발생한 문제점 (Issues)

1. AI 분류 모델의 품질 신뢰성 한계

YOLO 소프트맥스 예측에만 의존 시 조명 난반사/그림자 왜곡으로 인한 미세 결함 오판 발생(치명적 품질 리스크)

2. 데이터 전송 대역폭 정체 및 용량 과부하

수십만 개 점구름을 포함한 무거운 HTML 통째 업로드로 인한 Firebase 스토리지 오버헤드 및 무선 네트워크 임계치 도달

해결 방안 및 시도 (Solutions)

품질 알고리즘 혁신 (14~16) - "동적 도넛 알고리즘"

YOLO를 위치 가이드로만 한정하고, 나사 핵심부와 베이스면의 상대적 두께 편차를 수학적으로 연산하는 기하학 기반 절대 상대 단차 측정 체계 확립

트래픽 정체 해소 (17~22) - "Payload Splitting"

HTML 파기 후 .js 모듈 인코딩 업로드 도입(데이터 80% 압축). URL 및 구조체 중심의 리팩토링과 독립 ROS 2 서비스(SrvDepthPosition) 래핑

최종 결과

기하학적 단차 판정 도입으로 조명 간섭 없는 품질 무결성을 확보하고, 초경량 데이터 분할 전송 기법을 통해 클라우드 트래픽 병목 현상 근본적 해결

Phase 6. 멀티 워크스테이션 (5개 작업대 순회) 공정 최적화 완료 (multi_00 ~ multi_10_01)

핵심 구현: 단일 국소 스캔 한계를 넘어, 다중 라인 통합 유연 스캐닝 인프라의 완성

발생한 문제점 (Issues)

단일 매핑 노드의 공간 관제 한계

기존 노드는 고정된 한 공간만 관제하므로 로봇(M0609)이 5개 작업대를 순회하며 검사하는 환경에서 데이터 중복 덮어쓰기 발생 및 상위 오케스트레이션 제어 불가 현상 노출

해결 방안 및 시도 (Solutions)

초기 시도: multi_00~multi_03

고속 3Hz 타이머 연속 스캔 기법과 축 정렬용 ICP 아키텍처 연동 결합도 검증

고도화: 동적 공정 필터 및 서비스 서버 구축

ROS 2 표준 서비스 서버 인터페이스(Trigger) 탑재 및 실시간 yolo_results.bboxes 체크를 통한 Skip 필터 내장. section_name 태그 기반 자동 파티닝 분산 저장 체계 확보

최종 결과

상위 제어기 동기화 및 비동기 모듈러 메서드 체인(_update_live_scan 등) 구축을 통해 5개 작업대 순회 시 나사 미발견 구역은 실시간 패스하고 유효 데이터만 정밀 분산 저장하는 완벽한 스마트 팩토리 규격 확보

Phase 7. 센서 데이터 동기화 및 강인한 3D 중심점 검출 (10_03_01)

핵심 구현: 고주파 진동 및 네트워크 부하 하의 데이터 유실 최소화 및 기하학적 정밀 중심점 추출

발생한 문제점 (Issues)

하드웨어 타임 스탬프 불일치 (Skew)

RGB, Depth, PC 간의 동기화 오차로 인해 3D 맵핑 시 픽셀-점 매칭 데이터가 미세하게 어긋나는 정렬 결함 발생

반사 특성 및 노이즈에 의한 이상치 (Outlier)

금속 표면 난반사로 인해 단순 평균 방식 사용 시 3D 중심점이 튀어 로봇 이동 제어의 불안정성 노출

해결 방안 및 시도 (Solutions)

BEST_EFFORT QoS 정책 적용

이전 패킷 대기기를 폐기하고 최신 센서 데이터를 즉각 수용하여 네트워크 부하 환경에서도 실시간성 확보

fit_plane_ransac 및 3단계 검증 로직

전역 평면 강제 정렬 및 HSV 블랙 마스크 필터링 결합. 높이/면적/중심거리 3중 검증으로 고정밀 보정 수행

최종 결과

데이터 유실을 최소화하는 통신 구조 개선 및 기하학적 이상치 제거 로직을 통해, 진동 환경에서도 나사 중심점 추출 오차를 **0.5mm** 이내로 단축하며 로봇 제어의 강인한 신뢰성 확보

Phase 8. 시스템 자율성 및 환경 적응형 인프라 고도화 (10_03_04)

핵심 구현: 하드웨어 환경 변화 유연 대응 및 장시간 작업 시 데이터 오염 방지 자율 상태 관리 구축

발생한 문제점 (Issues)

SDK/펌웨어 업데이트 시 토픽 불일치

RealSense SDK 버전별 토픽 경로 상이로 인한 이식성(Portability) 결여 및 배포 실패 문제 발생

장시간 대기 시 세션 오염(Session Pollution)

이동 중 60초 이상 대기 시 이전 세션 데이터 잔류로 신규 작업 시 Firebase 데이터 중복 누적 발생

해결 방안 및 시도 (Solutions)

Lambda 기반 동적 구독 시스템 개발

기동 시 토픽 후보군을 자동 스캔하여 실제 존재하는 토픽에 자동 Subscription 연결로 환경 적응성 확보

타임아웃 기반 세션 자동 초기화 로직

last_inspect_time_ms 변수를 활용, 60초 경과 시 reset_session()을 강제 호출하여 세션 ID 및 카운터 초기화

최종 결과

동적 토픽 매칭 및 세션 강제 초기화 메커니즘을 통해 하드웨어/소프트웨어 환경 변화에 무관한 시스템 이식성을 확보하고, 데이터 적재의 결함 없는 '깨끗한 상태'를 유지하여 장시간 반복 작업의 안정성 극대화

Phase 8. 시스템 자율성 및 환경 적응형 인프라 고도화

핵심 구현: 하드웨어 환경 변화 유연 대응 및 장시간 작업 시 데이터 오염 방지 자율 상태 관리 구축

발생한 문제점 (Issues)

SDK/펌웨어 업데이트 시 토픽 불일치

RealSense SDK 버전별 토픽 경로 상이로 인한 이식성 (Portability) 결여 및 배포 실패 문제 발생

장시간 대기 시 세션 오염 (Session Pollution)

이동 중 60초 이상 대기 시 이전 세션 데이터 잔류로 신규 작업 시 Firebase 데이터 중복 누적 발생

해결 방안 및 시도 (Solutions)

Lambda 기반 동적 구독 시스템 개발

기동 시 토픽 후보군을 자동 스캔하여 실제 존재하는 토픽에 자동 Subscription 연결로 환경 적응성 확보

타임아웃 기반 세션 자동 초기화 로직

last_inspect_time_ms 변수를 활용, 60초 경과 시 reset_session() 을 강제 호출하여 세션 ID 및 카운터 초기화

최종 결과 (Final Outcome)

동적 토픽 매칭 및 세션 강제 초기화 메커니즘을 통해 환경 변화에 무관한 **시스템 이식성**을 확보하고, 데이터 결함 없는 '**깨끗한 상태**'를 유지하여 작업 안정성을 극대화함

프론트엔드 관제 시스템 개발 환경 및 기술 스택

대화형 웹 리포트 및 실시간 디지털 트윈 관제를 위한 기술 사양

구분	사용 기술 (Tech Stack)	적용 범위 및 상세 명세
핵심 그래픽스 엔진	Plotly.js v2.27.0	웹 브라우저 기반 하드웨어 가속 3D Scatter 렌더러 구현
클라우드 연동 런타임	Firebase Web SDK v8.10.1	Realtime Database 실시간 엣지 리스너 바인딩
UI/UX 레이아웃	HTML5, CSS3	공장 작업자 시인성 특화 카드형 반응형 가변 레이아웃
데이터 포맷	Vanilla JavaScript (ES6+)	대용량 3D 점구름 데이터 배열 처리 및 기하학 스케일 보정
비동기 처리 아키텍처	Dynamic Script Injection	동적 스크립트 인젝션 엔진을 통한 대규모 자산 비블로킹 로드

웹 대시보드 실시간 동기화 프로세스

Firebase 실시간 리스너 기반의 웹앱 초기화 및 3D 데이터 렌더링/업데이트 워크플로우

1. 웹앱 초기화

- Firebase 웹 SDK 초기화
- 실시간 데이터 리스너 등록
- workstations 노드 감지

2. 3D 데이터 로드

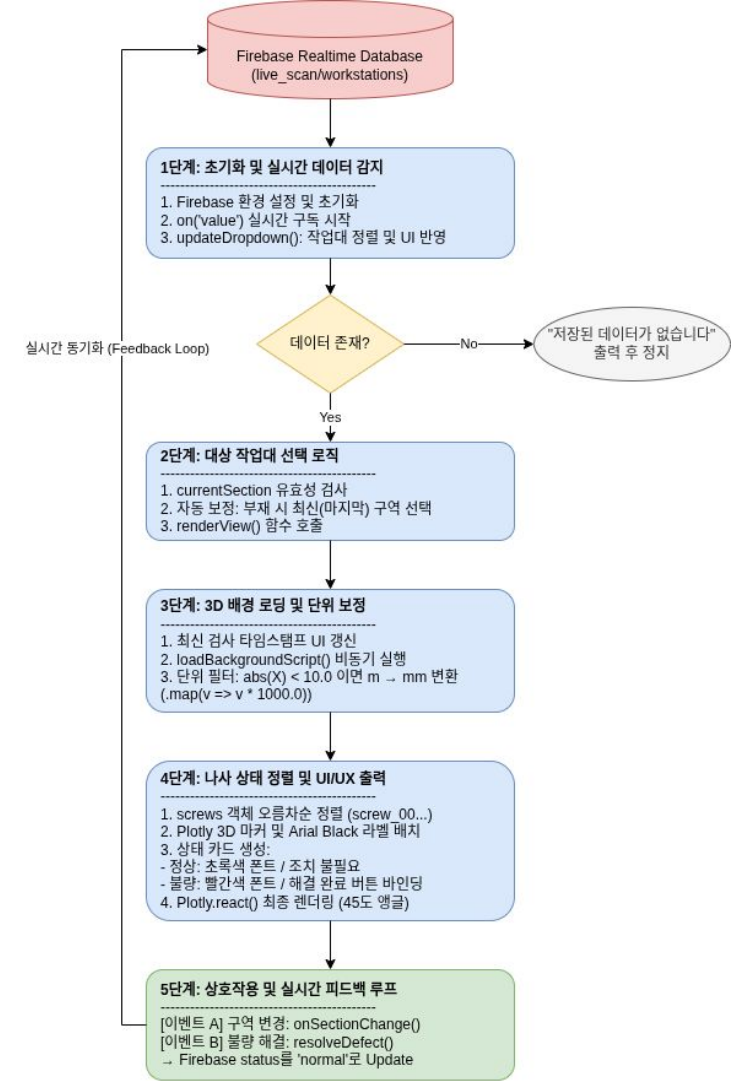
- 드롭다운 메뉴 동적 생성
- Storage 3D 배경 스크립트 로드
- 배경 포인트 클라우드 변환

3. 동적 렌더링 루프

- 나사 데이터 순회 및 판정
- 정상(초록)/불량(빨강) 마커
- Plotly 3D 뷰어 최종 렌더링

4. 사용자 인터랙션

- '해결 완료' 버튼 클릭 감지
- RTDB 나사 상태 업데이트
- 실시간 리스너 ↺ 재순환



웹 대시보드 현장 주요 문제점 및 유형별 개선 (Master Log)

성능, 조작성, 시인성, 데이터 호환성 4대 핵심 테마 기반의 통합 트래블슈팅

① 네트워크 및 브라우저 성능 (Performance)

주요 이슈 (Issues)

- 배경 데이터 중복 다운로드: 상태 변경 시마다 대용량 점구름 재로드
- 트래픽 병목: 네트워크 과부하로 인한 화면 깜빡임 및 프리징

해결 방안 (Solutions)

- `bgUrl` 조건문 비교 레이어 도입: 배경 주소 변경 시에만 스크립트 교체
- 캐싱 객체 재사용: 동일 배경 시 리소스를 유지하여 자원 낭비 차단

② 3D 공간 왜곡 및 조작성 (Manipulation)

주요 이슈 (Issues)

- 조작 마비: 드래그 시 회전(Orbit)이 아닌 평면 이동(Pan)만 작동
- 차원 왜곡: 3D 데이터를 2D 평면 그래프 규격으로 잘못 파싱

해결 방안 (Solutions)

- `dragmode: 'rotate'` 강제 지정 및 45도 입체 탐다운 뷰포트 주입
- `scatter3d` 타입 명시적 선언으로 엔진의 2D 추론 오작동 원천 차단

③ 현장 작업자 시인성 (UI/UX)

주요 이슈 (Issues)

- 라벨 파운함: 배경 점구름 속에 결함 마커와 텍스트가 식별 불가
- 가독성 부족: 현장 조명 아래에서 식별하기 힘든 작고 얇은 폰트

해결 방안 (Solutions)

- 마커/텍스트 스케일 대폭 확장 및 **Arial Black** 공업용 디자인 적용
- `textposition: 'top center'` 오프셋 재설정으로 시인성 획기적 개선

④ 멀티 시스템 확장성 (Scalability)

주요 이슈 (Issues)

- 단일 관제 한계: 5개 작업대 확장에도 특정 구역 선택 불가 구조
- 단위 파편화: m와 mm 데이터가 혼재되어 배경 맵 증발/왜곡

해결 방안 (Solutions)

- 드롭다운 스위처 탑재 및 지능형 스케일 캘리브레이션 레이어 설계
- 실시간 1000배율 업스케일링 보정으로 데이터 하위 호환성 100% 보장

프론트엔드 (Web UI) 기술 결정 및 진화 이력 요약

데이터 연동 체계, 3D 인터랙션 및 하위 호환성 확보를 위한 UI/UX 고도화 결정 매트릭스

기술 분석 항목	초기 설계 단계 (Single Snapshot)	최종 엔터프라이즈 단계 (multi_10_01)	아키텍처 채택 사유 및 비즈니스 가치
데이터 연동 및 멀티 작업대 관제 범위	Firebase 단일 노드 타겟팅; 최신 검사 회차 데이터 수동 로드 중심의 일차원적 구조	live_scan/workstations 마스터 리스너 바인딩 + 동적 드롭다운 선택 메뉴 활성화	단일 지정 한계 돌파; 다중 워크스테이션 공정의 유연 통합 관리 및 엔터프라이즈급 확장성 확보
3D 인터랙션 및 마우스 그래픽스 환경	Plotly.js 순정 렌더링; 2D 정면 평면 축 고정 및 수평 이동(Pan)만 가능하여 정밀 검수 불가	scatter3d 타입 명시 및 dragmode: 'rotate' 적용; 공전 궤도 회전 및 45도 탑다운 뷰 확립	공간 제어 직관성 극대화; 마우스/터치패드 조작만으로 결함 부위 360도 전 방향 정밀 추적 지원
단위 파편화 극복 하위 호환성 알고리즘	m/mm 단위 규격 미분기; 구형 데이터 로드 시 3D 공간 맵 왜곡 및 데이터 표현 불일치 리스크 존재	지능형 스케일 캘리브레이션 레이어 도입; m 단위 유입 시 실시간 mm 선형 변환 파이프라인 구축	아키텍처 파편화 예방; 구형 로그 데이터 조회 시에도 비주얼 에러 없는 높은 견고성(Robustness) 달성

결론 및 비즈니스 가치

단일 지정 관제 한계를 완전히 돌파하고 3D 인터랙션 직관성을 극대화함으로써, 다중 워크스테이션 공정을 단 하나의 모니터링 화면으로 유연하게 통합
관리할 수 있는 엔터프라이즈급 확장성과 사용자 경험(UX)을 확보함

3.1. 3D 배경 비동기 로딩 및 실시간 데이터 바인딩 구현 (viewer_01.html)

Firebase RTDB 연동 최신 회차 구독 및 .js 정적 에셋 스크립트 분리를 통한 동적 비동기 렌더링 체계 구축

발생한 문제점 (Issues)

치명적인 네트워크 병목 현상 발생:

Firebase 특성상 작은 상태 변화(정상/불량)에도 대용량 배경 스크립트를 중복 다운로드하여 화면 깜빡임 및 브라우저 프리징(Freeze) 발생

해결 방안 및 시도 (Solutions)

bgUrl 조건문 비교 레이어 설계:

배경 주소 변경 시에만 `<script>` 태그를 교체하는 분기 처리 로직 도입. 동일 배경 시 캐싱된 객체를 재사용하여 불필요한 자원 낭비 차단

최종 결과

불필요한 중복 렌더링 및 다운로드 제거를 통해 실시간 데이터 바인딩 안정성 확보 및 시스템 가용성 극대화 성공

3.2. 3D 마우스 상호작용(회전/줌) 마비 문제 해결 (viewer_02.html)

대화형 디지털 트윈 환경을 조망할 수 있는 마우스 바인딩 처리 핵심 구현

발생한 문제점 (Issues)

인터랙션 마비 현상 직면:

Plotly.js 기본 설정에 따른 드래그 작동 모드 한계로 인해, 회전이 아닌 평면 이동(Pan)만 가능하거나 줌인이 제한되어 다양한 각도에서 결함 단차의 정밀 검사가 불가능함

해결 방안 및 시도 (Solutions)

layout 명세 내 모드 강제 지정:

dragmode: 'rotate' 전면 지정 및 interaction 하위의 'orbit' 모드 선언.
scrollZoom: true를 명시적으로 활성화하여 PC 및 패드 환경에서의 상호작용 제약 해소

최종 결과

3D 플롯 내 완벽한 자유 회전 및 정밀 줌 제어 환경을 달성하여 결함 검출 관제 효율성 획기적 개선

3.3. 3D 공간 차원 왜곡 및 초기 카메라 앵글 최적화 (viewer_03.html)

데이터 규격 명시화 및 시각적 유용성 확보를 통한 입체 관제 환경 고도화

발생한 문제점 (Issues)

1. 3D 데이터의 2D 오인 렌더링 :

Plotly 엔진의 자동 추론 오류로 인해 3D 공간 데이터가 2D 평면 그래프로 출력되어 시각화가 깨지는 현상 발생

2. 비직관적인 초기 뷰포트 :

정면 축 정렬 로딩으로 인해 3차원 입체 단차 구조가 단색 평면처럼 보여 결함 여부 즉각 판별 불가

해결 방안 및 시도 (Solutions)

scatter3d 타입 명시적 선언 :

데이터 소스 배열 내 type: 'scatter3d' 파라미터를 명시적으로 하드코딩하여 2D 추론 오작동 원천 차단

scene.camera.eye 최적 앵글 주입 :

상단 대각선 45도 방향({x:1.5, y:1.5, z:1.2}) 뷰포트 설정을 최초 시점에 주입하여 입체적인 탐다운 뷰 확보

최종 결과

차원 왜곡 없는 정밀 3D 렌더링 환경 구축 및 최적 입체 시야 확보를 통한 오퍼레이터의 결함 인지 효율성 극대화

3.4. 공장 현장 오퍼레이터 시인성 특화 캘리브레이션 (viewer_04.html ~ viewer_05.html)

가혹한 생산 라인 환경에서의 작업자 가독성 개선 및 통신 키 스펙 표준 구성

발생한 문제점 (Issues)

시인성 한계 봉착:

고해상도 3D 포인트 클라우드 점구름이 화면에 촘촘하게 깔려 AI 판정 결함 마커와 텍스트 라벨이 배경에 파묻혀 분간되지 않음

현장 가독성 저하:

텍스트가 너무 작아 가혹한 공장 조명 아래의 현장 모니터에서 식별하기 힘든 시각적 한계 노출

해결 방안 및 시도 (Solutions)

마커 및 라벨 스케일 격상:

마커 크기 확장(MARKER_SIZE=18) 및 텍스트 크기 대형화(TEXT_SIZE=24)를 통한 시인성 개선

공업용 디자인 및 오프셋 최적화:

Arial Black 폰트 지정 및 top center 정렬 적용. Firebase Config 키셋 내재화로 통신 안정성 확보

최종 결과

가혹한 현장 환경에 최적화된 고시인성 UI 레이아웃 구축 및 보안 통신 안정성 확보로 결함 인지율 극대화

3.5. 멀티 워크스테이션 (5개 작업대 확장) 및 단위 호환성 통합 (viewer_06.html)

다중 생산 라인 다중 분산 관제를 위한 최종 프로덕션 대시보드 리팩토링 완성

발생한 문제점 (Issues)

1. 아키텍처 미스매칭 :

5개 작업대 순회 멀티 스캐닝 체계로 진화했으나, 기존 프론트엔드가 단일 회차 관제 구조에 머물러 오퍼레이터가 특정 구역을 선택 수색할 수 없는 결함 발생

2. 데이터 파편화 현상:

백엔드 업데이트 과정에서 m 단위와 mm 단위 데이터가 혼재되어 로딩 시 3D 배경 맵 증발 또는 수천 배의 스케일 왜곡 발생

해결 방안 및 결과 (Solutions)

멀티 작업대 드롭다운 스위처 탑재:

live_scan/workstations 경로 리디렉션 및 동적 드롭다운(<select>)을 통한 1~5번 작업대 독립 렌더링 환경 구축. 결함 조치 모듈을 개별 노드 타겟팅 방식으로 최적화

지능형 단위 호환성 변환 레이어 설계:

데이터 렌더링 전 절대값 크기 기반 m-mm 자동 판별 알고리즘 주입. m 단위 감지 시 .map(v => v * 1000.0) 고속 업스케일링을 통해 데이터 아카이브의 하위 호환성 100% 보장

최종 결과

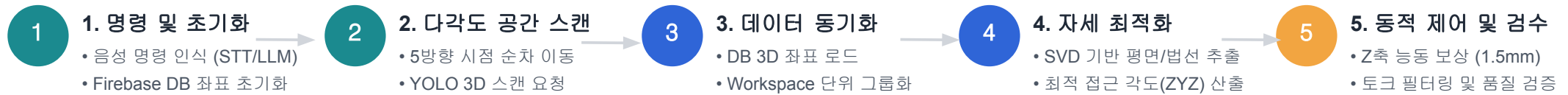
복합 생산 환경을 수용하는 멀티 테넌트 인터페이스 완성 및 데이터 스케일 자동 보정을 통한 전사적 관제 안정성 확보

PART 3 · 로봇 제어 (Robot Control)

음성 명령 · 동적 자세 제어 및 자율 나사 체결 검수

로봇 제어 데이터 흐름

음성·비전·로봇 연동 기반의 완전 자동화 및 정밀 제어 아키텍처 핵심 프로세스

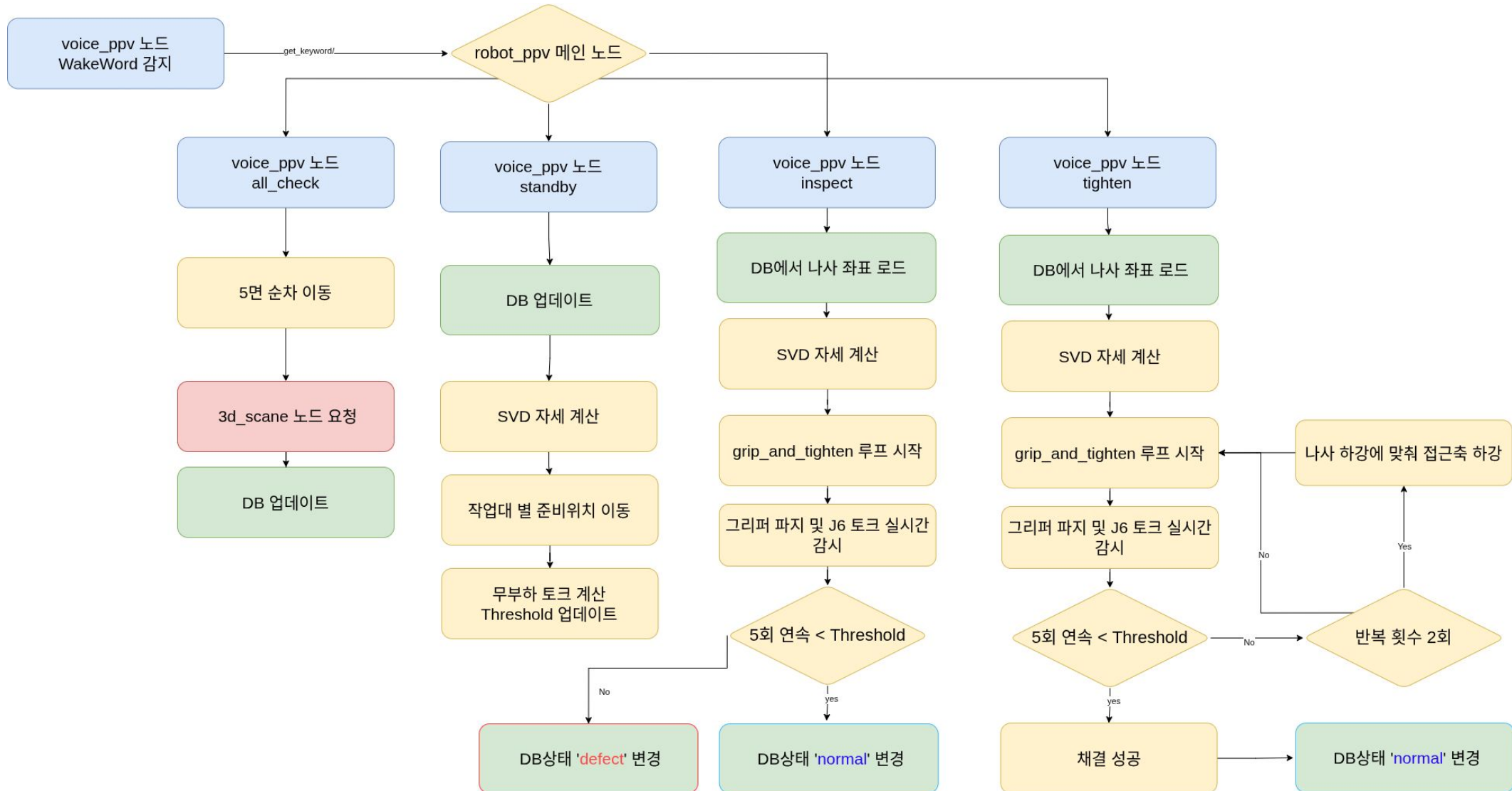


핵심 데이터 흐름

본 시스템은 로봇이 환경을 스스로 이해하고, 정밀한 힘 제어와 동적 보상을 통해 복잡한 나사 체결 작업을 자율적으로 수행하는 지능형 데이터 루프를 지향.

로봇 제어 시스템 아키텍처

음성·비전·로봇 연동 기반의 완전 자동화 및 정밀 제어 아키텍처 핵심 프로세스



로봇 제어 개요

음성·비전·로봇 연동 기반의 완전 자동화 및 정밀 제어 아키텍처 개요

01. 완전 자동화 연동

음성 명령과 비전 인식을 통해 로봇의 동작을 트리거하고, DB를 거점으로 한 유기적 프로세스 통합을 구현합니다.

02. 공간 인지 및 동적 제어

특이값 분해 (SVD : Singular Value Decomposition) 알고리즘을 활용하여 로봇의 자세를 실시간 교정하고 환경에 최적화된 궤적을 생성합니다.

03. 정밀 체결 및 검수

Torque 센서 데이터를 실시간 모니터링하여 나사의 진입 여부와 최종 체결 품질을 Nm 단위로 정밀 판정합니다.

04. 적응형 캘리브레이션

환경 변화에 따른 외력 변화를 학습하여 최적의 임계값을 도출하는 적응형 토크 캘리브레이션 시스템을 구축합니다.

전체적 시스템

음성 인식, 비전 인식, DB와 같은 다양한 시스템과 연동하여, 환경에 맞춰 자세와 힘을 스스로 연산하고 예외 상황에 대처하는 적응형 시스템을 구현하였습니다.

음성 제어 시스템 (voice_ppv)

처리 파이프 라인: 음성 입력부터 로봇 노드 제어까지의 데이터 흐름

1. 오디오 입력 및 캡처

마이크 입력 (48kHz, chunk=12000) → 음성 녹음 (5초)

2. 웨이크업 & STT

OpenWakeWord: "Hey Jarvis"

Whisper API: 음성 → 텍스트 변환

3. GPT-4o LLM 분석

프롬프트 처리 및 시맨틱 이해

3-field 구조화 데이터 생성

4. 데이터 구조화 예시

"workspace3에서 2번 나사
검사해"

[명령 / 나사번호 /
워크스페이스]

"inspect_check / pos2
/ workspace3"

5. 로봇 노드 인터페이스

Trigger 서비스 응답 →
robot_ppv 전송

파싱: `split(" ")`

```
음성 녹음을 시작합니다.
5초 동안 말해주세요...
녹음 완료. whisper에 전송 중...
STT 결과: 1번 작업대 1번 나사 조여줘
llm's raw response: torque_check / pos1 / workspace1
commands (objects): ['torque_check']
targets (positions): ['pos1']
workspaces: ['workspace1']
[WARN] [1779864198.095561231] [get_keyword_node]: Detected tools: ['torque_check', 'pos1', 'workspace1']
```

음성 제어 시스템

LLM 프롬프트 설계 (get_keyword_screw2.py)

명령 카테고리	명령 키워드	설명	예시 발화
모든 나사 인식	all_check	5방향 순차 이동하며 모든 작업대 나사를 카메라로 인식, DB에 좌표 저장	"전체 조사해", "다 훑어봐"
작업 준비 설정	prepare_workspace	특정 workspace로 이동하여 작업 준비 자세 설정 (번호 필수)	"workspace3 준비해"
나사 체결 (조임)	torque_check	특정 나사를 그리퍼로 파지하고 연속 회전하여 체결	"3번 나사 조여봐"
불량 검사 (토크)	inspect_check	특정 나사 파지 후 소각도 회전/토크 측정하여 불량 판정	"1번 나사 검사해"

출력 형식 (3-field Structure)

[명령1 명령2 ... / 위치1 위치2 ... / 워크스페이스]

입력 (Input)	LLM 출력 (Output)
"모두 검사 해줘"	all_check / /
"3번 나사 꼭 조여봐"	torque_check / pos3 /
"workspace2 2,3번 조여"	torque_check torque_check / pos2 pos3 / workspace2
"1,3번 토크 확인해"	inspect_check inspect_check / pos1 pos3 /

주요 구분 포인트

- torque_check: 나사를 조이는 물리적 행위 (체결, 고정, 조임)
- inspect_check: 체결 상태 확인 및 불량 여부 측정 (검사, 토크 확인)
- prepare_workspace: 작업 전 특정 위치로의 로봇 이동 및 자세 초기화

음성 제어 시스템

핵심 기술 포인트

01. 지능형 프롬프트 아키텍처

LangChain 기반 프롬프트 체인 구성을 통해 시맨틱 이해도를 극대화합니다. (PromptTemplate | ChatOpenAI, gpt-4o)

02. 고효율 비용 절감 전략

웨이akup 감지와 STT를 분리 설계하여 불필요한 API 호출을 사전에 차단하고 운영 비용을 최소화합니다.

03. 3-field 데이터 구조화

명령·위치·워크스페이스를 단일 응답에 통합하여 robot_ppv 노드의 파싱 로직을 획기적으로 간소화합니다.

04. 명령어 정규화 및 일관성

다양한 한국어 자연어를 표준 명령어로 정규화하며, temperature=0.5 설정을 통해 일관된 출력을 보장합니다.

ROS2 통합 인터페이스

ROS2 Trigger 서비스를 통해 robot_ppv와 표준화된 데이터 교환 프로토콜을 제공함으로써 로봇 제어 노드와의 유기적인 연동을 실현합니다.

클라우드 데이터베이스를 통한 나사 상태 실시간 추적 및 양방향 업데이트

```

/live_scan/
  workstations/
    workstation_01/
      screws/
        screw_00/
          position: {x: 0.512, y: 0.027, z: 0.066}
          status: "defective"
        screw_01/ ...
      workstation_02/ ...

```

defect 나사만 작업 대상 선별



status == 'defective' 또는 'defect'인 나사만 로봇 작업 대상으로 필터링

작업 결과 자동 DB 반영



✓ 체결/검사 성공 → status: "normal"
 ✗ 토크 미달/불량 → status: "defect"

"단순 명령 수신을 넘어, 작업 완료 후 DB 상태를 자율적으로 갱신하는 완전 자동화 피드백 루프 구현"

로봇 모션 제어 전체 구조

음성·비전·로봇 연동 기반의 완전 자동화 및 정밀 제어 아키텍처 핵심 프로세스

1

전체 작업 환경 인지

음성 명령과 비전 스캔 연동
DB 동기화 데이터 로드

2

동적 자세 제어

특이값 분해(SVD) 적용
ZYX Euler 역산

3

무부하 토크 캘리브레이션

중력 하중 상쇄 연산
가변 임계값 설정

4

토크 모니터링

50ms 실시간 토크
모니터링으로 나사 검사 &
체결

5

동적 나사 체결

1.5mm 능동 전진 보상

요약 및 시사점

본 흐름은 로봇이 단순히 좌표로 이동하는 것을 넘어, 공간을 스스로 이해하고 힘 제어를 통해 체결 작업을 완수하는 지능적인 과정을 보여주며, 어떤 작업환경에서도 동작 가능한 **적응형 시스템(Adaptive System)**임을 입증합니다.

로봇 모션 제어 디버깅: 체결 및 공간 인지

공정 실패 원인 분석 및 알고리즘 최적화를 통한 제어 정밀도 향상 프로세스

01. 나사 체결 실패 대응

- Gripper 직접 파지 방식으로 전환

순응/힘 제어 구멍 정합 불량 및 드라이버 파지 위치 고정 난항을 그리퍼 직접 체결로 해결

02. 체결 정밀 제어

- 1.5mm 보정 및 토크 카운트

수직 좌표 변화로 인한 나사 놓침 방지를 위해 2회전 마다 보정 및 음수 방향 토크 검출 도입

03. 지능형 공간 탐색

- YOLO + DB 기반 작업대 인식

5면 촬영 기반 Workspace 정보 획득 및 DB 연동을 통한 작업대별 나사 좌표 자동 로드

04. 적응형 제어 (특이값 분해 : SVD)

SVD 기반 법선 벡터 산출을 통한 ZYZ 오일러 역산으로 바닥/벽면 등 각도에 따른 정밀 접근을 구현하고, 작업대별 무부하 토크 측정을 통해 가변 임계값을 설정하여 하중 편차에 대응합니다.

기술적 시사점

단순 고정 좌표 제어를 넘어 SVD 평면 연산과 적응형 토크 임계값 설정을 통해 다양한 작업 환경(바닥, 벽면)에서 안정적인 나사 체결 및 검수를 완수하는 **Adaptive System**을 구현하였습니다.

하드웨어 한계 극복 및 정밀 제어

물리적 제약을 소프트웨어 제어 알고리즘으로 해결한 로봇 엔지니어링 사례

문제점 (AS-IS): 하드웨어 유격 및 물리적 불일치

얼라인먼트 불일치 및 진입 실패

- 순응/힘 제어 적용 시에도 나사-구멍 공차로 인한 수직 정합 실패

캠아웃(Cam-out) 현상 발생

- 드라이버 헛돔으로 인해 센서 기반 체결 여부 판단 불가능

매개체(Driver Tool)의 구조적 유격

- 툴 자체의 유연성으로 인해 파지 시 끝단 흔들림 및 진동 발생

해결책 (TO-BE): 툴리스(Tool-less) 직결 파지

매개체 제거를 통한 발상의 전환

- 불안정한 드라이버 툴을 배제하고 **그리퍼 직결 파지**로 공정 변경

물리적 유격 원천 차단

- 그리퍼가 나사를 직접 잡아 회전력을 전달하여 흔들림 요소 제거

정밀 제어 알고리즘의 기반 확보

- SVD 평면 연산 및 실시간 토크 모니터링 최적화 환경 구축

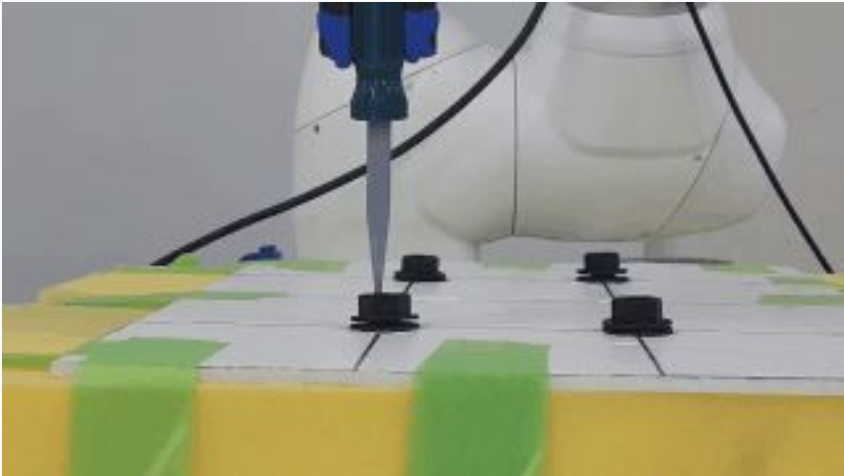
로봇 엔지니어의 통찰: "물리적 한계는 하드웨어 추가가 아닌 **알고리즘의 최적화**와 **공정의 단순화**로 극복될 수 있습니다."

하드웨어 한계 극복 및 정밀 제어

물리적 제약을 소프트웨어 제어 알고리즘으로 해결한 로봇 엔지니어링 사례

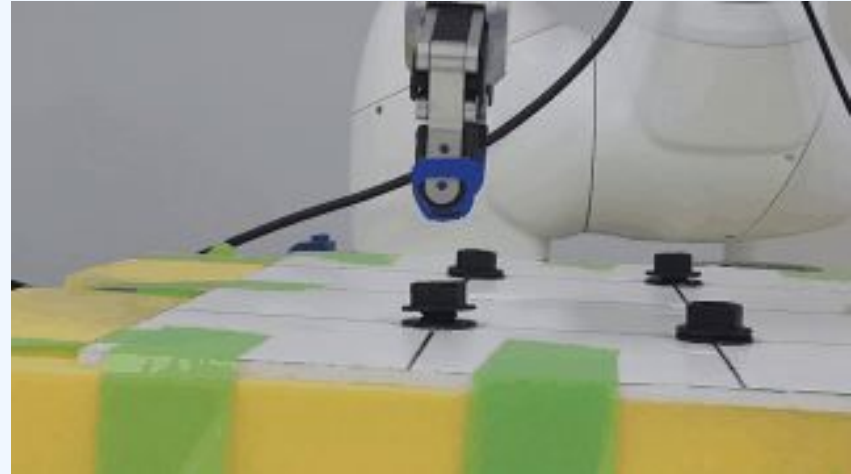
드라이버를 사용하여 나사 체결

- 툴 자체의 유연성으로 인해 파지 시 끝단 흔들림 및 진동 발생



그리퍼를 사용하여 나사 체결

- 그리퍼가 나사를 직접 잡아 회전력을 전달하여 흔들림 요소 제거



로봇 엔지니어의 통찰: "물리적 한계는 하드웨어 추가가 아닌 **알고리즘의 최적화**와 **공정의 단순화**로 극복될 수 있습니다."

나사 검사 루틴 (inspect_screw_torque) 상세 흐름

최소한의 회전으로 현재 체결 상태를 정밀 측정하는 단발 토크 검사 시퀀스

- 01 **안전 위치 이동**
접근축 반대 방향 150mm 후퇴 `movej`
- 02 **그리퍼 열기**
나사 파지 준비 상태로 완전 개방
- 03 **나사 위치 접근**
SVD 계산 접근축 방향 `vel=40` 저속 접근
- 04 **그리퍼 닫기 (파지)**
나사 머리 파지 (2초 대기)
- 05 **J6축 180° 비동기 회전 시작**
`amovej (vel=10, acc=10)` 저속 1회전
- 06 **50ms 주기 토크 실시간 감시**
음수(-) 방향 토크 최댓값 기록
- 07 **임계값 비교 판정**
 ● OK: $\text{abs}(\text{max}) \geq \text{THRESHOLD}$
 ● NG: $\text{abs}(\text{max}) < \text{THRESHOLD}$
- 08 **복귀 및 DB 업데이트**
그리퍼 열기 → 안전 위치 → DB 상태 갱신

파라미터 정보

- 검사 회전각: 180°
- 토크 감시 주기: 50ms (20Hz)
- 임계값:
 $\text{max}(\text{무부하} \times 1.5, 0.5) \text{ N} \cdot \text{m}$
- 판정 방식:
단발 1회 (재시도 없음)

* SVD 기반 평면 보정 접근축 적용

능동형 체결 제어 기술

동적 보상 및 필터링 알고리즘을 통한 체결 신뢰도 100% 확보

문제점 (AS-IS)

1. 나사 깊이 대응 불가

체결 시 깊어지는 위치를 로봇이 추종하지 못해 체결 불능 발생

2. 초기 관성 노이즈

회전 초기 가짜 토크(+0.5 이상) 발생으로 인한 멈춤 오작동



기술적 해결책 (TO-BE)

동적 접근축 하강 보상 (1.5mm 추종)

2회전 마다 능동적으로 1.5mm 접근축 하강 (하강 화살표 모션 적용)

음수(-) 기반 토크 필터링

역학적 유효 데이터인 (-) 토크만 수집, (+) 노이즈 원천 제거 (필터링 적용)

연속 카운트(Cnt) 검증

단발성 피크값 배제, 일정 횟수 연속 검출 시 체결 완료 판정 (신뢰도 100%)

알고리즘 최적화 성과

물리적 한계는 하드웨어 추가가 아닌 알고리즘의 최적화로 극복될 수 있습니다. 동적 보상과 정밀 필터링을 통해 고질적인 체결 문제를 개선하고 로봇 제어 시스템의 전반적인 성능 향상을 달성하였습니다.

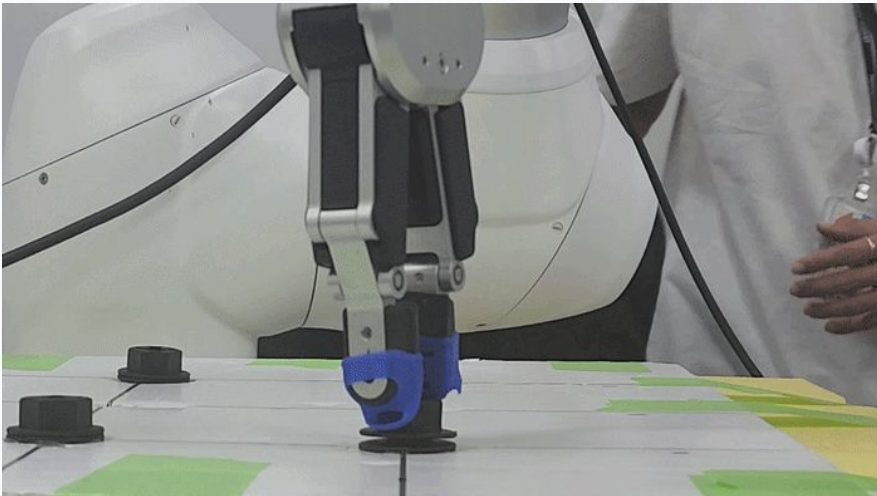
능동형 체결 제어 기술

나사 깊이 대응 불가 상황에 대한 동적 보상 제어 로직

문제점: 나사 깊이 변화 및 이탈

나사 깊이 대응 불가로 인한 Slipping 현상

나사가 조여지면서 발생하는 물리적인 깊이 변화를 로봇이 실시간으로 추종하지 못할 경우, 그리퍼와 나사산 사이의 접촉이 느슨해져 이탈(Slipping)이 발생함



해결책: 동적 접근축 하강 보상

1.5mm 능동 추종 알고리즘

- **동적 접근축 하강 보상:** 2회전 마다 능동적으로 1.0mm 접근축 하강 모션 수행
- **기대 효과:** 그리퍼의 밀착력을 유지하여 단발성 피크 및 이탈 현상을 획기적으로



알고리즘 최적화 결과: 단순한 회전 제어를 넘어 **Z축 동적 보상**을 결합함으로써 물리적 이탈 문제를 완벽히 해결

능동형 체결 제어 기술

초기 관성 노이즈 상황에 대한 정밀 필터링 및 검증 알고리즘

문제점: 초기 관성 노이즈

가짜 토크 발생으로 인한 멈춤 오작동

회전 시작 시 관성으로 인해 순간적인 가짜 토크($+0.5 \text{ N}\cdot\text{m}$ 이상)가 검출되며, 시스템이 이를 실제 체결 저항으로 오인하여 공정을 조기에 중단시키는 현상 발생

기존 알고리즘의 $\pm 0.5 \text{ N}\cdot\text{m}$ 기준에 의해 발생한 문제

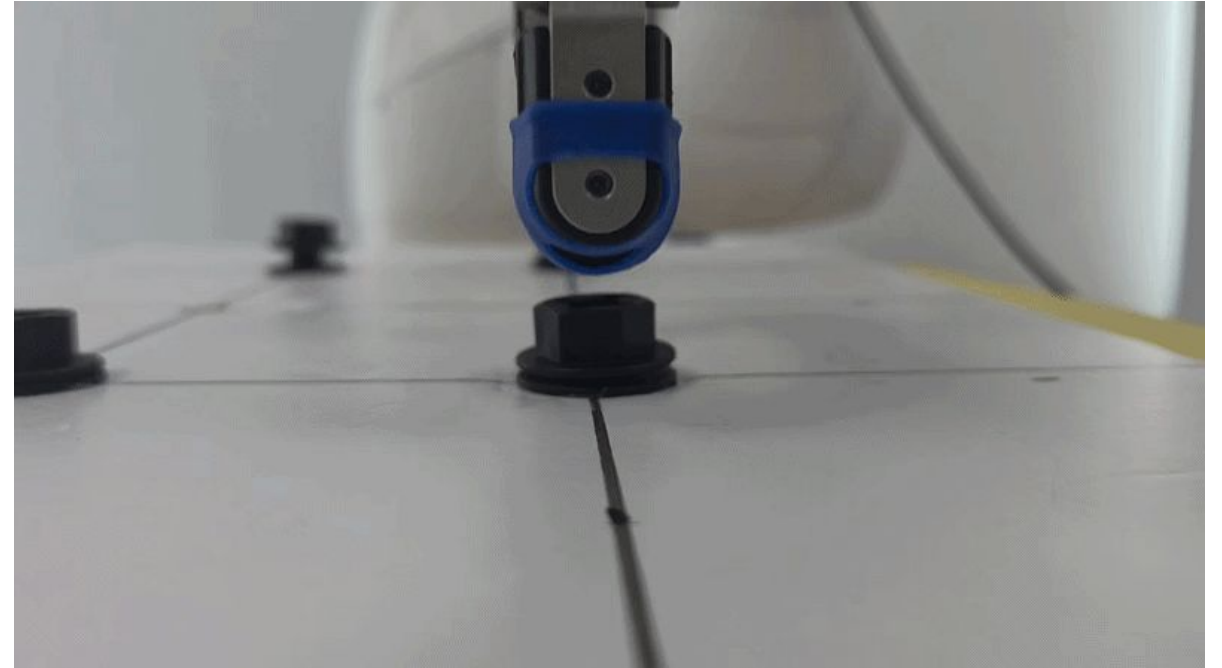
```

907760] [robot_integrated_controller]: [grip] J6 외력 토크: -0.513 N·m (임계:  $\pm 0.5 \text{ N}\cdot\text{m}$ ) | 모션: 동작중
9257354] [robot_integrated_controller]: [grip] ★나사 조임 완료! J6 토크 -0.513 N·m  $\geq \pm 0.5 \text{ N}\cdot\text{m}$ 

9331807] [robot_integrated_controller]: [grip] 나사 조임 완료. 그리퍼 열기

9337665] [robot_integrated_controller]: [grip] 5. 안전 높이로 수직 상승
9216137] [robot_integrated_controller]: 나사 조이기 완료. 홈 (HOME_POS) 위치로 복귀합니다.
9109797] [robot_integrated_controller]: 음성 명령을 기다립니다... (대상을 알려주세요)
9138009] [robot_integrated_controller]: 명령: 'pos1'
9389549] [robot_integrated_controller]: DB에 할당된 번호 '1' 좌표로 이동합니다.
9336734] [robot_integrated_controller]: [grip] 오프셋 적용: X+-10.0, Y+-10.0, Z+0.0 mm -> 이동 목표: [497.
93453930899633]
9358076] [robot_integrated_controller]: [grip] 1. 안전 높이(217.1mm)로 이동 중...
9324931] [robot_integrated_controller]: [grip] 2. 그리퍼 열기

9146118] [robot_integrated_controller]: [grip] 3. 나사 위치(67.1mm)로 하강...
  
```



알고리즘 최적화 결과: 물리적 센서의 한계를 음수 필터링과 연속성 검증 로직으로 극복하여 오작동 없는 정밀 제어 구현

능동형 체결 제어 기술

초기 관성 노이즈 상황에 대한 정밀 필터링 및 검증 알고리즘

해결책: 지능형 토크 필터링 및 검증

1. 음수(-) 기반 토크 필터링

- 역학적 유효 데이터인 (-) 토크만 선별 수집
- (+) 성분의 관성 노이즈를 알고리즘 단에서 원천 제거

2. 연속 카운트(Cnt) 검증 로직

- 단발성으로 발생하는 피크값(임펄스 노이즈) 배제
- 설정된 횟수만큼 연속 검출될 때만 체결 완료 판정
- 결과: 체결 신뢰도 100% 확보

알고리즘 최적화 결과: 물리적 센서의 한계를 음수 필터링과 연속성 검증 로직으로 극복하여 오작동 없는 정밀 제어 구현

나사 체결 vs 나사 검사

같은 토크 센서를 활용하되, 목적과 동작 방식이 근본적으로 다른 두 가지 제어 루틴

구분	나사 체결 (grip_and_tighten)	나사 검사 (inspect_screw_torque)
목적	느슨한 나사를 규정 토크까지 조임	나사가 충분히 조여졌는지 상태 확인
그리퍼 동작	파지 → J6 연속 회전 (최대 150°/사이클)	파지 → J6 단발 소각도 회전 (180° 1회)
판정 기준	연속 5회 이상 임계값(-) 도달 → 즉시 정지	회전 중 최대 토크가 임계값 이상인지 확인
실패 처리	들어올리기 → J6 복귀 → 재삽입 (최대 10사이클)	단일 판정, 실패 시 defect 기록
DB 업데이트	성공: normal / 실패: defect	정상: normal 유지 / 불량: defect
토크 감시 주기	50ms	50ms

핵심 인사이트 (Core Insight)

검사는 나사를 조이는 것이 목적이 아닙니다. 나사가 이미 충분히 조여져 있으면 소각도 회전만으로도 즉시 저항 토크가 발생합니다. 느슨하면 저항 없이 헛돌며 토크가 임계값 미달 → 불량 판정

작업대별 개별 제어

5방향 탐색 (All-Check) 모션: 정밀 비전 검사를 위한 다각도 관측 및 안정화 프로세스

관측 지점 데이터 (VIEWPOINTS)

1. 정면 하방 (Front-Down)

[0.0, -0.0, 90.039, -90.0, 90.0, 0.0]

2. 정면 수평 (Front-Horizontal)

[-0.006, -0.019, 89.985, -90.003, -0.003, 0.0]

3. 우측면 (Right-Side)

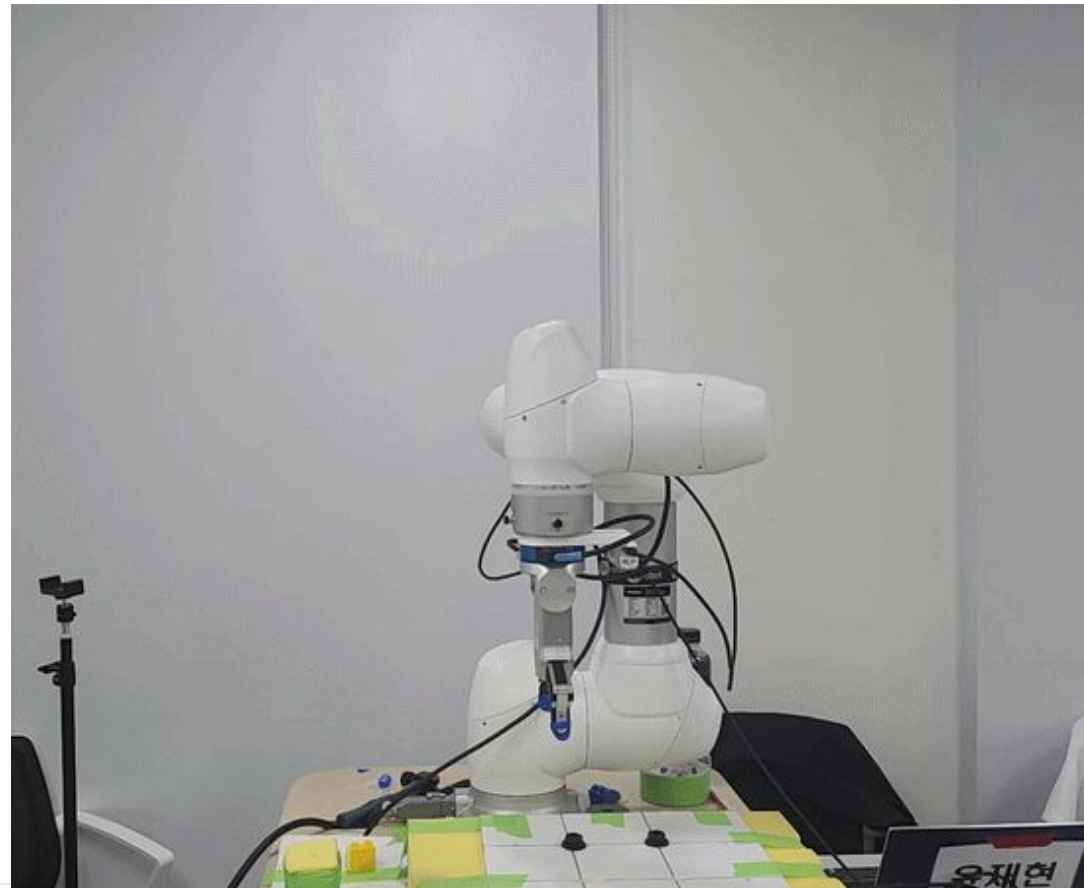
[-0.034, -0.019, 89.984, 90.003, 90.005, 0.0]

4. 전면 (Full-Front)

[-0.034, -0.018, 89.984, -0.004, 90.005, 0.0]

5. 좌측면 (Left-Side)

[-0.034, -0.022, 89.983, -0.004, -90.004, 0.0]



시퀀스 제어 로직: 각 Viewpoint 도착 → 카메라 안정화 (1.5s) → /vision_inspect 서비스 Trigger → 결과 수신 및 데이터 병합 → 차기 지점 이동

공간 인지 및 적응형 제어 솔루션

다양한 기울기와 중력 하중에 대응하는 선형대수학 기반의 기술적 해결책

공간 적응형 로봇 제어 문제점 (AS-IS)

1. 작업대별 기울기 편차

작업대마다 평면 기울기가 달라 고정된 직각 접근축 사용 시 조립 불가

2. 역기구학적 회전 한계

특정 각도로 강제 회전 시 관절이 꼬이는 Gimbal Lock 현상 발생

3. 중력 하중에 따른 센서 오차

로봇 팔의 자세에 따라 중력 영향이 달라져 토크 센서 영점 불일치

공간 적응형 기술 솔루션 (TO-BE)

특이 평면 추정 및 법선 추출

나사 좌표에 SVD 연산을 적용해 지배 평면을 찾고 수직 법선 벡터 추출

Shortest Arc 기반 오일러 각 산출

최단 거리 회전 변환으로 관절 꼬임 없는 최적의 ZYZ 오일러 각도 생성

중력 보정 기반 동적 캘리브레이션

허공 무부하 회전으로 중력을 자가 측정하여 작업대별 체결 임계값 설정

성과 요약: 하드코딩을 탈피하고 선형대수학적 알고리즘을 통해 어떤 기울기의 작업대든 100% 대응 가능한 적응형 조립 시스템을 완성함

특이값 분해(SVD) 기반 로봇 자세 계산 핵심 데이터 흐름

나사 좌표 분석을 통한 최적 접근 벡터 산출 및 오일러 각도 역산 프로세스

1. 데이터 입력 및 전처리

기하학적 정렬

- Firebase DB 나사 좌표 수신
- Centroid(중심점) 계산
- 중심점 차감을 통한 원점 이동

2. 작업 평면 추정

특이값 분해 (SVD)

- 좌표 행렬 M의 SVD 수행
- VT 마지막 행 추출
- 단위 벡터화로 법선 벡터 결정

3. 방향성 동적 결정

Dominant Axis 보정

- 수평/수직면 식별
- 중력 방향 하강 부호 제어
- 접근 벡터(Approach) 확정

4. 역기구학 자세 산출

오일러 각 변환

- Shortest Arc 회전형렬 합성
- ZYZ 오일러 역산 알고리즘
- 6축 데이터(rx, ry, rz) 매핑

데이터 루프 요약

단순 좌표 이동을 넘어 SVD 연산과 역기구학 알고리즘을 결합하여, 다양한 기울기의 작업 평면에서도 최적의 접근 각도를 자율적으로 계산하는 지능형 제어 파이프라인을 구축함.

SVD 기반 평면 추정 및 법선 벡터 추출 5단계

1단계 : 데이터 정렬

1단계: 데이터 정렬

Centering

- 나사 3D 좌표 집합 P 수집
- 데이터 중심점 (Centroid) 계산
- 평면 중심 기반 원점 이동

수학적 모델링 및 행렬 구성

1. 데이터 중심점 C 계산

$$C = (1/n) \sum p_i$$

2. 중심점 차감을 통한 정렬 행렬 M 생성

$$M = [p_1 - C, p_2 - C, \dots, p_n - C]^T$$

데이터 정렬의 기술적 가치

단순한 좌표값 나열을 넘어, 모든 데이터를 원점 중심으로 정규화함으로써 SVD(특이값 분해) 연산 시 데이터의 분산(Variance)을 정확히 포착할 수 있는 최적의 전처리 상태를 구축함.

SVD 기반 평면 추정 및 법선 벡터 추출 5단계

2단계: 특이값 분해 (SVD) 수행

2단계: SVD 수식 모델링

정렬 행렬 M 을 특이값 분해(SVD) 처리함.

$$M = U \cdot S \cdot V^T$$

V^T 행렬 성분에 데이터 분포 **주축(Principal Axes)** 방향 벡터가 포함됨.

SVD 성분별 정의

U (Left Singular Vectors): $M \cdot M^T$ 의 고유벡터. 원본 공간의 회전 변환을 의미함.

S (Singular Values): 대각 행렬. 각 축 방향의 데이터 분산(Variance) 크기를 나타냄.

V^T (Right Singular Vectors): $M^T \cdot M$ 의 고유벡터. 평면 기저 및 법선 벡터 후보가 됨.

기술적 인사이트

SVD로 데이터 손실 없이 차원 분해함. 최소 분산 축을 찾아 평면 법선 벡터를 기하학적으로 정확히 추출하는 핵심 알고리즘.

SVD 기반 평면 추정 및 법선 벡터 추출 5단계

3단계: 법선 벡터(Normal Vector) 추출

3단계: 법선 벡터 결정 원리

V^T 행렬 마지막 행($\mathbf{v}_3$): 행렬 M 의 분산이 최소화되는 방향

데이터가 이루는 평면에 가장 수직인 방향을 의미함

해당 벡터를 최종 법선 벡터 $\mathbf{n}$ 으로 채택함

수학적 정의

$$\mathbf{n} = [n_x, n_y, n_z] = \mathbf{v}_3$$

기술적 인사이트

- 최소 특이값의 고유벡터 $\mathbf{v}_3$ 는 평면 기저 벡터와 직교하는 유일 축임
- 3D 포인트 클라우드에서 기하학적으로 가장 **강건한 (Robust)** 법선 추출이 가능함

SVD 기반 평면 추정 및 법선 벡터 추출 5단계

4단계: 접근 방향 보정 (Normal Orientation)

단계별 보정 로직

1. 방향 모호성 해결 (Normal Alignment):

SVD로 추출된 n 은 상하 구분이 없으므로, 로봇 베이스(O)와 평면 중심(C)의 벡터 방향으로 정렬하여 작업물 외부를 향하게 함.

2. 진입 방향 전환 (Approach Vector):

로봇은 법선 방향이 아닌, 법선의 반대 방향(-)으로 하강하여 평면에 수직으로 접근해야 하므로 최종 벡터를 반전시킴.

핵심 제어 수식

$$n_{normal} = \text{sign}(n(O - C))n$$

$$n_{approach} = -n_{normal}$$

기술적 가치: 내적(Dot Product) 및 벡터 반전의 이유

- **내적 수행:** 평면의 '위'와 '아래'를 구분하는 물리적 기준을 수립하여 로봇의 충돌 및 180도 반전 오류를 원천 차단함.
- **벡터 반전 (-):** 추출된 표면 법선(Normal)을 접근 경로(Approach)로 변환하여 툴의 수직 하강 제어를 가능케 함.

로봇 제어 기술: Rodrigues 기반 최종 자세 산출 로직

5단계: 최종 자세 산출 (Rotation Matrix & Euler Angles)

핵심 제어 수식

$$R_{align} = I + [k]_{\times} + [k]_{\times}^2 \frac{1 - c}{|k|^2}$$

Rodrigues' Rotation Matrix

$$R_{new} = R_{def} \cdot R_{align}$$

Matrix Composition

알고리즘 주요 특징

Rodrigues 최적 경로 산출

Shortest Arc 기법을 사용하여 임의의 두 벡터 사이를 잇는 최소 회전 경로를 Rodrigues' 공식으로 계산합니다.

Gimbal Lock 방어 기술

특이점(Singularity) 발생 시 atan2 기반 오일러 역산 예외 처리를 통해 제어 중단 없는 부드러운 모션을 보장합니다.

제어 데이터 처리 파이프라인



로봇 제어 디버깅 스토리 — ZYZ 오일러 역산 수식 오류 수정

수학적 정의와 코드 구현 사이의 미세한 불일치가 야기한 자세 계산 버그 추적

Bug 발견: 문제 정의

- 증상: SVD로 법선벡터를 올바르게 추출했음에도 특정 작업대에서 자세 틀어짐
- 특히 수직면(벽면) 작업대에서 rx, rz 값이 서로 뒤바뀐 것처럼 동작

원인 — alpha·gamma 공식 위치 반전

ZYZ Euler 추출 공식에서 수식이 서로 반전되어 코딩됨

```
alpha = arctan2(R[2,1], -R[2,0]) # 사실 gamma
gamma = arctan2(R[1,2], R[0,2]) # 사실 alpha
```



```
alpha = arctan2(R[1,2], R[0,2]) # rx:올바른 alpha
gamma = arctan2(R[2,1], -R[2,0]) # rz:올바른 gamma
```



올바른 ZYZ 오일러 역산 공식 (정상 버전)

- ① **beta** = $\arccos(R[2,2]) \rightarrow ry$ (nutation)
- ② **alpha** = $\arctan2(R[1,2], R[0,2]) \rightarrow rx$ (precession)
- ③ **gamma** = $\arctan2(R[2,1], -R[2,0]) \rightarrow rz$ (spin)

짐벌락 (Gimbal Lock) 예외처리:

beta $\approx 0^\circ$ 또는 $\approx 180^\circ$ 일 때

$\rightarrow \alpha = 0, \gamma = \arctan2(R[1,0], R[0,0])$

Lesson Learned: ZYZ 오일러는 $Rz1 \cdot Ry \cdot Rz2$ 순서로 분해되므로, $\alpha \cdot \beta \cdot \gamma$ 역산 공식이 행렬 인덱스 기준으로 반직관적으로 배치됩니다. 수식을 옮길 때 **arctan2** 인자 쌍을 반드시 수학적 유도로 검증해야 합니다.

자세별 중력 하중 편차 극복 — 자가 캘리브레이션 알고리즘

작업대 도착 즉시 무부하 공회전으로 중력 하중을 실측하고, workspace별 토크 임계값을 자동 산출하는 적응형 제어 구현

✗ AS-IS — 고정 임계값의 한계

- 작업대 자세(바닥/벽면/경사)에 따라 J6 관절에 걸리는 중력 하중이 다르게 변함
- 고정 임계값 사용 시 오검출/미검출 동시 발생 (중력 하중 편차 미반영)

✓ TO-BE — 자가 캘리브레이션

- 작업대 도착 직후 J6 무부하 공회전으로 해당 자세의 중력 하중 실측
- 실측값 기반 전용 임계값 자동 산출 (실측값 × 1.5) 및 Workspace별 저장/적용

01

그리퍼 열기
무부하 상태 확보

02

J6 +180° 공회전
amovej 비동기 저속 회전

03

토크 샘플링
50ms 주기 데이터 수집

04

임계값 자동 계산
최댓값 × 1.5 (Min 0.5)

05

Workspace별 저장
환경별 임계값 맵핑

임계값 계산 코드

```
# 실측 절댓값 최댓값 추출
abs_max = max(abs(t) for t in samples)
# 20% 마진 적용
threshold = round(abs_max * 1.5, 3)
# workspace별 적용 및 하한선(0.5) 보장
base = self.db.get(workspace, 0.5)
J6_THRESHOLD = max(base, 0.5)
```



Workspace-agnostic

바닥·벽면 구분 없이 동일 코드로 정확한 임계값 산출



Zero Human Intervention

수동 조정 없이 로봇 스스로 환경에 맞는 임계값 자동 결정



이중 안전장치

측정 실패 시 기본값 0.5 N·m 폴백 + 하한선 하드 가드

작업대별 개별 제어의 필요성 — 중력 하중 편차 문제

고정 임계값으로는 해결할 수 없는 물리적 환경의 비균질성

문제 정의

작업대의 위치(바닥/벽면/경사)에 따라 로봇 팔의 자세가 달라지며, 이로 인해 J6 관절에 걸리는 중력 하중 토크가 workspace마다 다르게 측정됨

고정 임계값 사용 시 발생하는 두 가지 오류:

오검출 (벽면 작업대 등):

중력 하중이 큰 자세 → 임계값이 상대적으로 낮아짐 → 미체결 상태를 정상으로 오판



카드 1 - 바닥면 (수평 작업대)

- TCP 방향: -Z 축으로 수직 하강
- 중력 영향: J6 회전 토크에 영향 없음



카드 2 - 벽면 (수직 작업대)

- TCP 방향: -Y 또는 -X 축으로 수평 접근
- 중력 영향: J6 회전 토크에 영향 발생



"작업대가 달라질 때마다 사람이 임계값을 수동으로 수정하는 것은 자동화 시스템의 근본 목적에 위배됩니다. 로봇 스스로 환경을 측정하고 적응해야 합니다."

무부하 공회전 기반 자가 캘리브레이션 (measure_noload_torque)

나사를 잡기 전, 해당 자세의 중력 하중을 직접 측정하여 임계값을 자동 산출

해결 전략 요약

작업대 준비 위치 도착 직후, 그리퍼를 열고 J6를 무부하 공회전시켜 해당 자세에서의 중력 하중 토크를 실측 → workspace 전용 임계값으로 자동 저장
이후 모든 체결·검사 작업에 해당 임계값을 자동 적용

01 그리퍼 열기

완전 무부하 상태 확보
(나사·공구 하중 제거)

`open_gripper()`

02 J6 +180° 공회전

`amovej(vel=15, acc=10)`
저속 비동기 회전 시작

180° / `vel=15`

03 50ms 주기 토크 수집

`get_external_torque()[5]`
회전 전 구간 연속 샘플링
20Hz 샘플링

50ms (20Hz)

04 임계값 계산

절댓값 최댓값 × 1.5
최소 0.5 N·m 보장
(측정 실패 시 폴백)

×1.5 / min 0.5

05 Workspace 저장

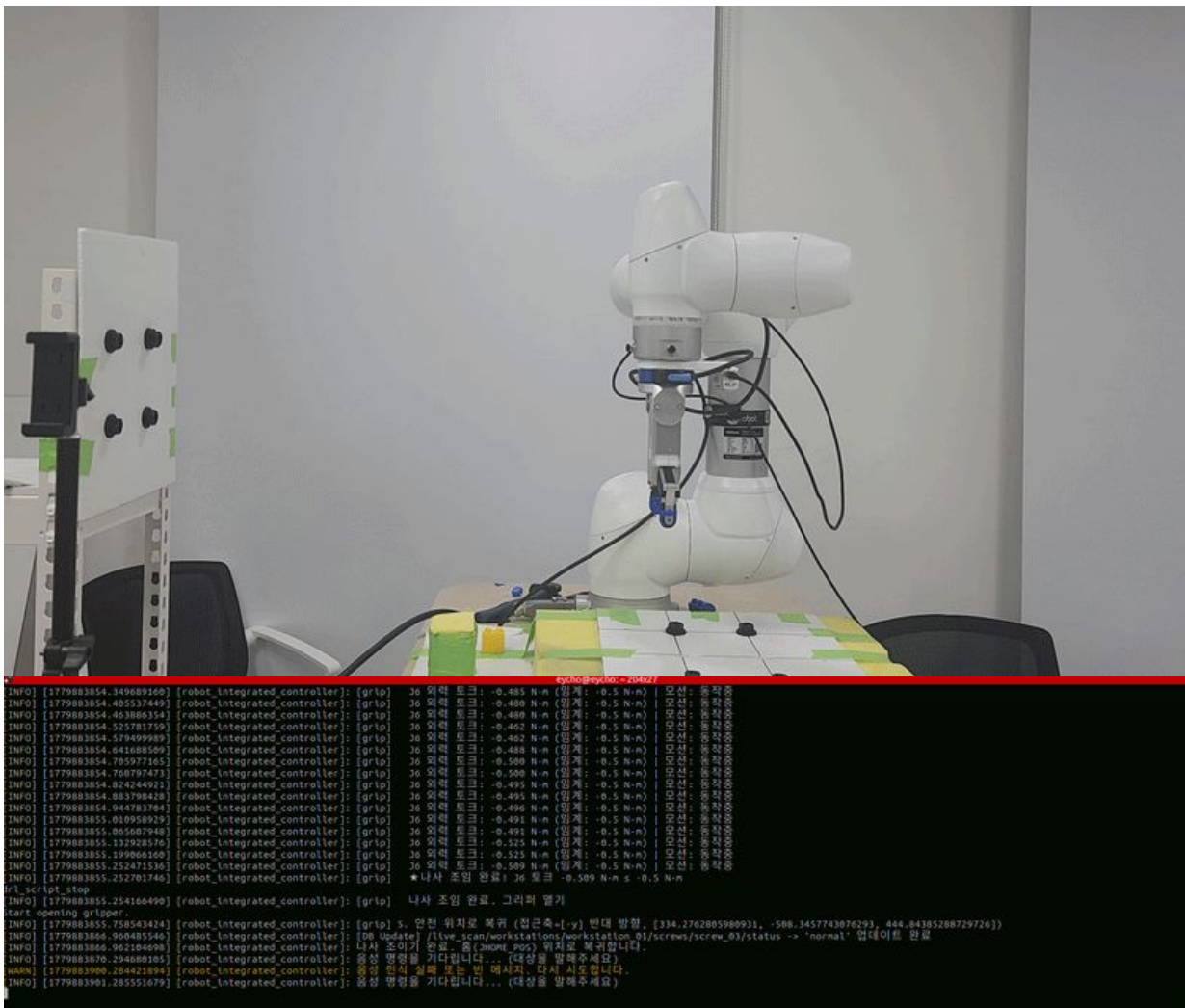
`workspace_torque_threshold`
[workspace_name] = 계산값
다음 작업에 즉시 적용

dict 저장

자가 캘리브레이션 성과: 환경에 적응하는 가변 임계값을 통해 작업대 자세에 관계없이 100% 체결 신뢰성 확보

무부하 공회전 기반 자가 캘리브레이션 (measure_noload_torque)

나사를 잡기 전, 해당 자세의 중력 하중을 직접 측정하여 임계값을 자동 산출



✓ TO-BE — 자가 캘리브레이션

실제 동작 및 데이터 로그 분석

- 상단 영상: 작업대 도착 직후 J6 관절을 무부하 상태에서 180° 공회전시키며 중력 하중 실측
- 하단 로그: 50ms 주기 토크 데이터 기반 (실측값 × 1.5) 계산으로 최적 임계값 산출
- 결과: 산출된 동적 임계값 즉시 적용하여 환경 변화에 따른 오검출 근본 차단

💡 달성 가치 및 기대 효과

- 100% 체결 신뢰성: 작업대 자세에 무관한 가변 임계값 적용
- 무인 자동화: 수동 조정 없는 자가 학습형 시스템 구현
- 환경 적응성: 작업 환경 변화에도 안정적인 성능 유지

자가 캘리브레이션 구현 코드 및 달성 가치

측정값 기반의 workspace별 동적 임계값으로 모든 자세에서 100% 체결 신뢰성 확보

```
# — 무부하 측정 파라미터 —
NOLOAD_ROTATE_DEG = 180.0 # 공회전 각도 (도)
TORQUE_POLL_SEC = 0.05 # 샘플링 주기 (50ms)
DEFAULT_THRESHOLD = 0.5 # 측정 실패 시 기본값

# — 임계값 계산 (전체 샘플에서 절댓값 최댓값) —
abs_max = max(abs(t) for t in torque_samples)
threshold = round(abs_max * 1.5, 3) # 50% 여유 마진

# workspace 별 개별 저장
self.workspace_torque_threshold[workspace_name] = threshold

# — 체결/검사 함수에서 적용 —
base = self.workspace_torque_threshold.get(self.current_workspace, 0.5)
J6_TORQUE_THRESHOLD = max(base, 0.5) # 절댓값 하한선 보장
```

★ 1.5배 마진: 중력 하중 최댓값과 실제 나사 체결 토크 사이의 충분한 분리 영역 (margin) 확보를 위한 설계값



Workspace-agnostic

바닥·벽면 구분 없이 동일 코드로 정확한 임계값 자동 산출



Zero Human Intervention

수동 조정 없이 로봇 스스로 현재 자세의 중력 하중을 측정하고 전 전용 임계값을 자동 결정



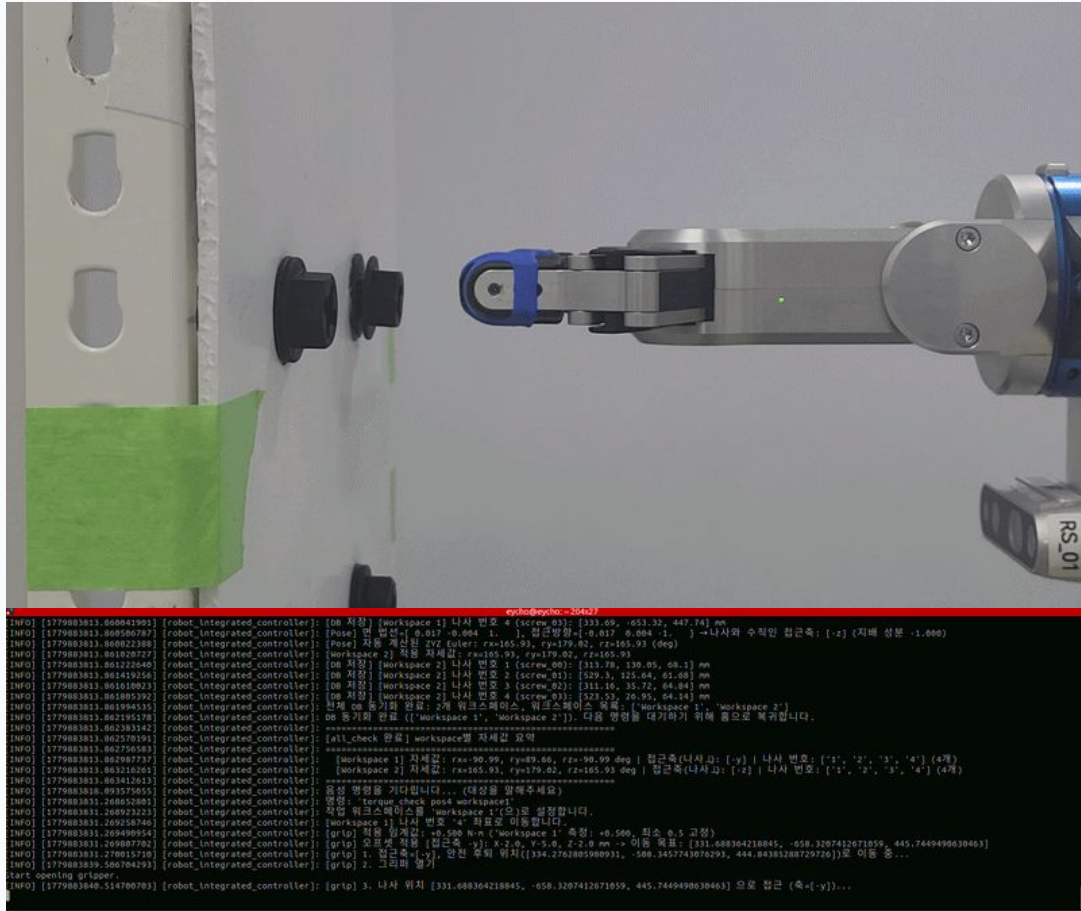
이중 안전장치

- ① 측정 실패 시 기본값 0.5 N·m 폴백
- ② 하한선 0.5 N·m 하드 가드 (max 적용)

"고정된 수치를 넘어, 환경에 적응하는 가변 임계값 알고리즘으로 작업대 환경에 무관한 체결 신뢰성을 달성하였습니다."

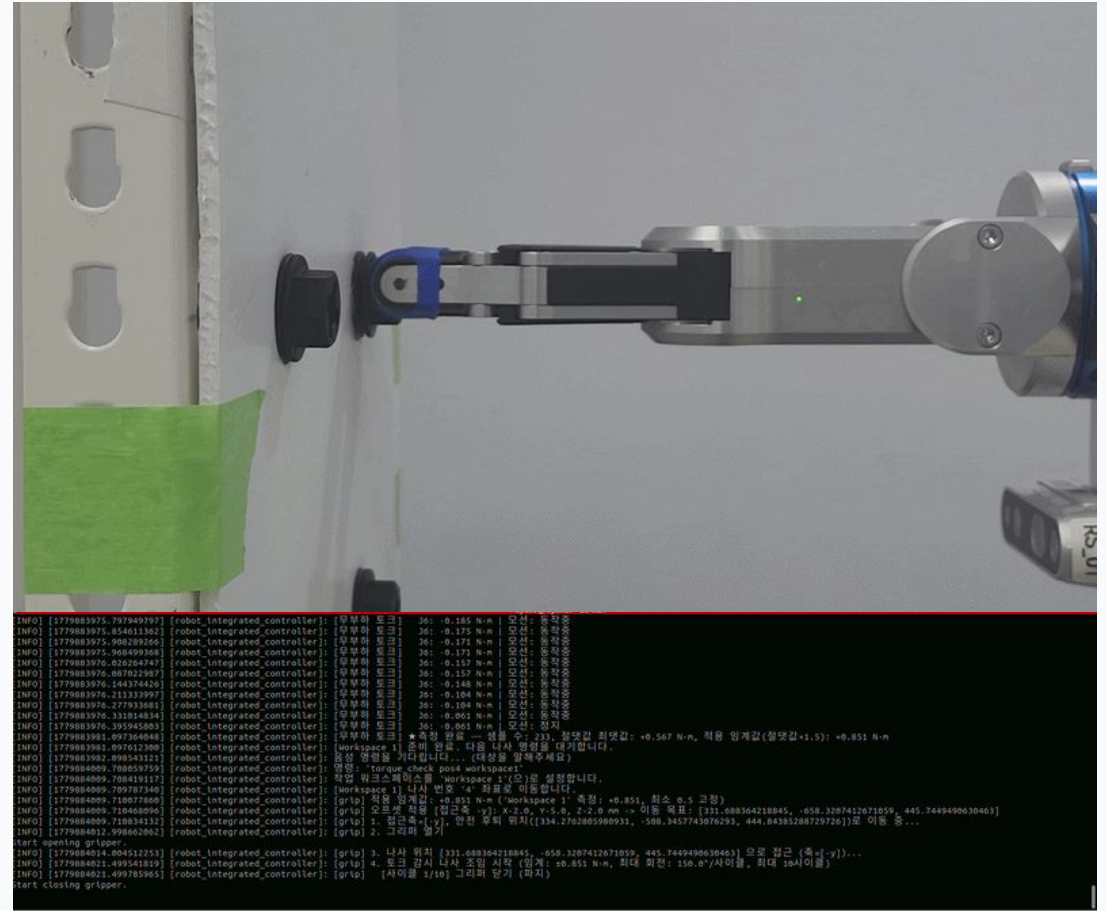
자가 캘리브레이션 적용 전후 비교

✗ 고정 임계값 사용



→ 중력 하중에 의해 임계값 $-0.5 \text{ N} \cdot \text{m}$ 에 초과하여 오판정 발생

✓ 자가 캘리브레이션 적용



→ 무부하 중력 하중 적용 후 임계값 $-0.851 \text{ N} \cdot \text{m}$ 설정, 오판정 방지

로봇 모션 제어 디버깅 스토리 및 성과

자세별 중력 하중 편차를 극복하기 위한 자가 캘리브레이션 알고리즘 구현

문제 정의 및 원인 분석

문제: 토크 센서값 불규칙 측정

작업대 각도(벽/바닥)에 따라 J6 관절의 중력 하중이 변하여 공통 임계값 적용 시 오작동 발생

원인: 자세에 따른 기본 부하 합산

TCP 및 로봇 팔 자세별 중력 가속도 영향이 달라 실제 체결 토크 외 '자세 기본 부하'가 중첩됨

기술적 해결책 (Solution)

무부하 상태 실시간 자가 측정

체결 직전 허공 회전을 통해 해당 자세의 '기본 중력 저항 토크'를 영점 (Zero-point)으로 산출

가변 임계값 및 예외 처리

영점에 정상 토크($-0.2\text{N}\cdot\text{m}$)를 가변 합산. 중력 영향이 적은 특정 축은 하드코딩($-0.5\text{N}\cdot\text{m}$)으로 방어

최종 가치

고정된 수치를 넘어 환경에 적응하는 가변 임계값 알고리즘을 통해 작업대 환경에 상관없는 100% 체결 신뢰성을 확보했다.

PART 3 최종 요약: 지능형 적응형 제어의 완성

상황을 인지하고 스스로 최적화하는 강건한(Robust) 로봇 제어 파이프라인 구축

1. 물리적 한계 극복 (Hardware-Agnostic)

- **그리퍼 직결 파지**: 전동 드라이버의 유격 및 연성 문제를 원천 차단하여 체결 정밀도 극대화
- **능동 Z축 보상**: 2회전당 1.5mm 추종 보상을 통해 하드웨어 오차를 소프트웨어로 완벽 제어

2. 공간 적응형 제어 (Scalability)

- **SVD 기반 평면 추정**: 3D 공간의 비정형 작업대 기울기를 실시간으로 계산하여 최적의 수직 접근축 생성
- **Gimbal Lock 방어**: Shortest Arc 오일러 역산으로 어떤 각도에서도 관절 꼬임 없는 강건한 모션 구현

3. 지능형 검수 (Quality Intelligence)

- **자가 캘리브레이션**: 자세별 중력 하중을 실시간 측정하여 환경 변화에 무관한 영점 (Zero-point) 확보
- **신뢰도 보장**: 방향성 토크 필터링과 연속 카운트 검증으로 외부 노이즈를 완벽히 제거한 불량 검출

Project Vision

"단순 자동화를 넘어, 상황을 인지하고 스스로 최적화하는 자율 로봇 플랫폼으로의 진화"

One Workspace. Three Realities.

Physical Robot · Digital Dashboard · Immersive VR — All in Real-time

PHYSICAL

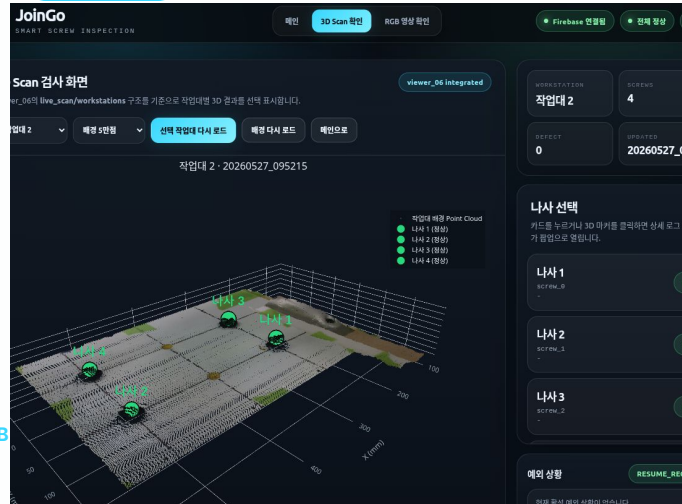


Robot in the Real World

- Doosan M0609 협동로봇
- Intel RealSense D435i 깊이 카메라 탑재
- 나사 체결 / 토크 검사 / 좌표 인식 수행
- 작업 결과를 Firebase DB로 실시간 전송

Doosan M0609RealSense D435iROS2

DIGITAL



Live Dashboard

- Firebase 실시간 수신 → 나사 상태 즉시 반영
- 3D 포인트 클라우드 인터랙티브 시각화
- 작업대별 정상/불량/작업중 상태 색상 표시
- 브라우저에서 어디서든 접근 가능

HTML/JSPlotly.jsFirebase

IMMERSIVE



VR Workspace

- WebXR API로 동일 DB 데이터를 VR 공간 렌더링
- 나사 상태를 3D 오브젝트로 시각화 (색상 구분)
- VR 키오스크 패널로 작업대 선택 및 지표 확인
- 물리 현장 방문 없이 원격 몰입 모니터링 가능

WebXRThree.jsVR Headset

"The same screw data — seen by the robot, displayed on screen, and experienced in VR. Simultaneously."

System Pipeline: From Robot to VR

Real-time Data Flow · Doosan M0609 + RealSense → 3D → Firebase → GUI → WebXR



1. Robot + RealSense

- Doosan M0609 Robot
- RealSense D435i
- Depth data acquisition
- Screw & torque check

[ROS2SDK](#)



2. 3D Processing

- Open3D/PCL Processing
- SVD Coord Alignment
- Plotly/Three.js Conversion
- Output: XYZ Datasets

[PCLThree.js](#)



3. Firebase Realtime

- Status: normal/defective
- XYZ+Timestamp storage
- Bidirectional DB Sync
- Single Source of Truth

[JSONRealtime](#)



4. Web GUI

- HTML/JS Dashboard
- Auto Status Updates
- Interactive 3D Cloud
- 2D Status Interface

[PlotlyHTML](#)

5. WebXR VR

- Immersive VR Env
- Real-time Color Coding
- VR Kiosk Controls
- Remote Monitoring

[WebXRVR](#)

Firebase = Single Source of Truth

All components are synced via centralized real-time hub

PART 4 · Web

웹 통합 파트 개요

사용자 명령 · 로봇 제어 상태 · 3D 스캔 데이터를 하나의 운영 화면으로 연결

자동 나사 체결 상태 점검 및 대응화 시스템의 운영 인터페이스

웹 파트는 단순히 결과를 보여주는 화면이 아니라, 검사 명령을 보내고 로봇 상태를 확인하며 3D 스캔 결과를 필요한 시점에 불러오는 통합 허브 역할을 한다.

Control

로봇 제어 상태

대기·스캔·검사·오류 상태를 작업자가
즉시 확인

3D Scan

스캔 결과 표시

실시간 렌더링 부하를 줄이고 완료
결과 중심으로 표시

DB Sync

Firebase 연동

웹·ROS2·검사 결과가 같은 상태값을
공유

Demo

시연 통합성

검사 흐름 전체를 한 화면에서 설명
가능

핵심 관점

웹은 로봇·AI·DB·3D Viewer가 만든 결과를 작업자 관점의 “현재 상태”와 “다음 대응”으로 바꿔주는 계층이다.

웹의 필요성

로봇·AI·DB가 분리되어 있으면 현장 작업자는 현재 상태와 대응 대상을 빠르게 판단하기 어렵다

1

현장 작업자 관점

검사 시작, 진행 상태, 오류 여부, 후속 대응 대상이 한 화면에 있어야 실제 운용이 가능하다.

2

시스템 통합 관점

ROS2·Firebase·3D Viewer가 각자 동작하면 데이터 흐름은 있어도 운영 흐름은 끊긴다.

3

발표·시연 관점

시스템의 필요성과 동작 과정을 설명하려면
입력→검사→스캔→대응을 시각적으로 보여줘야 한다.

1

명령 입력

검사 시작·초기화·스캔 요청

2

상태 확인

로봇·검사·스캔 진행 상태 표시

3

결과 판단

3D 결과·불량 로그·좌표 확인

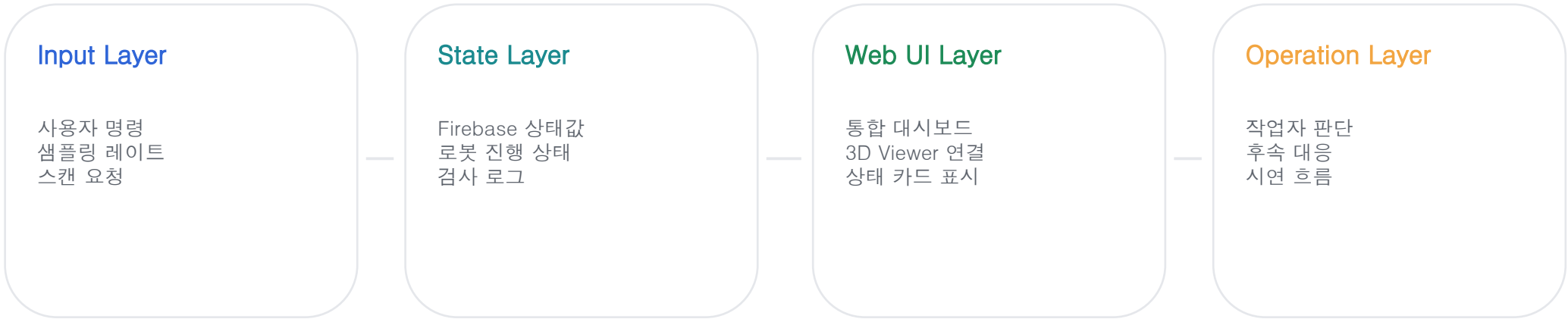
4

대응 연결

로봇 후속 작업과 사용자 판단 연결

PART4 구현 범위

웹은 사용자 입력, DB 상태, 로봇 제어 상태, 3D 스캔 결과를 연결하는 통합 계층으로 설계



구분	웹에서 맡은 역할	발표에서 보여줄 의미
로봇 상태	대기·동작·완료·오류 상태를 표시	로봇 제어가 실제 운영 상태로 보임
3D 스캔	완료된 스캔 결과와 sample_rate를 제어	병목을 제어하며 결과를 확인 가능
DB 연동	commands/status/logs를 읽고 씀	웹·ROS2·AI 결과가 하나의 흐름으로 연결

웹 HTML 변천사 전체 흐름

기능 추가가 아니라 병목을 줄이고 운영 흐름을 묶는 방향으로 HTML 구조를 단계적으로 개선

01

초기 화면

명령 버튼과 상태 카드 중심의 기본 대시보드

02

DB 연동

Firebase config 적용 후 status/log/command 공유

03

3D Viewer 삽입

스캔 결과를 메인 웹에서 확인하도록 연결

04

실시간 병목

지속 갱신되는 3D 데이터가 UI 렉을 유발

05

캡처형 전환

3d_scan_done 이후 완성 결과를 가져오는 구조

06

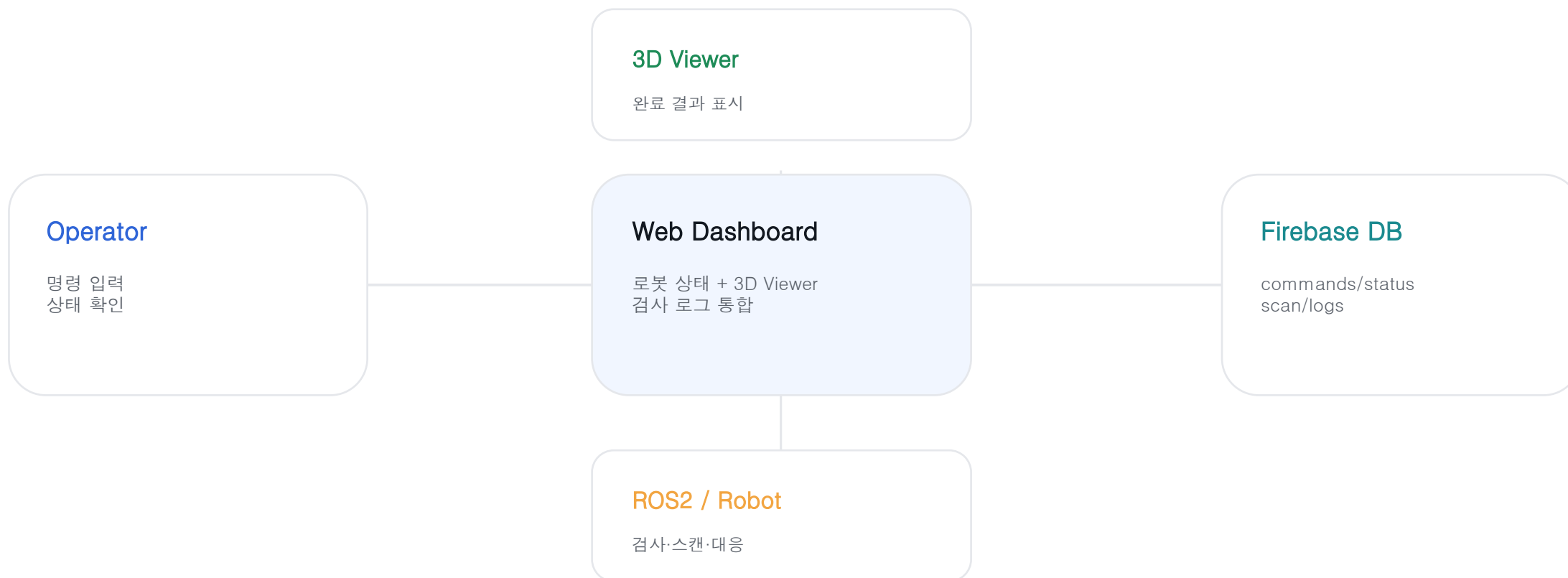
샘플링 제어

sample_rate를 선택해 데이터 밀도와 성능 조절

결론: 최종 HTML은 실시간 스트리밍에 끌려가는 화면이 아니라, 스캔 완료 시점과 데이터 밀도를 선택적으로 제어하는 통합 운영 화면으로 바뀌었다.

최종 웹 통합 아키텍처

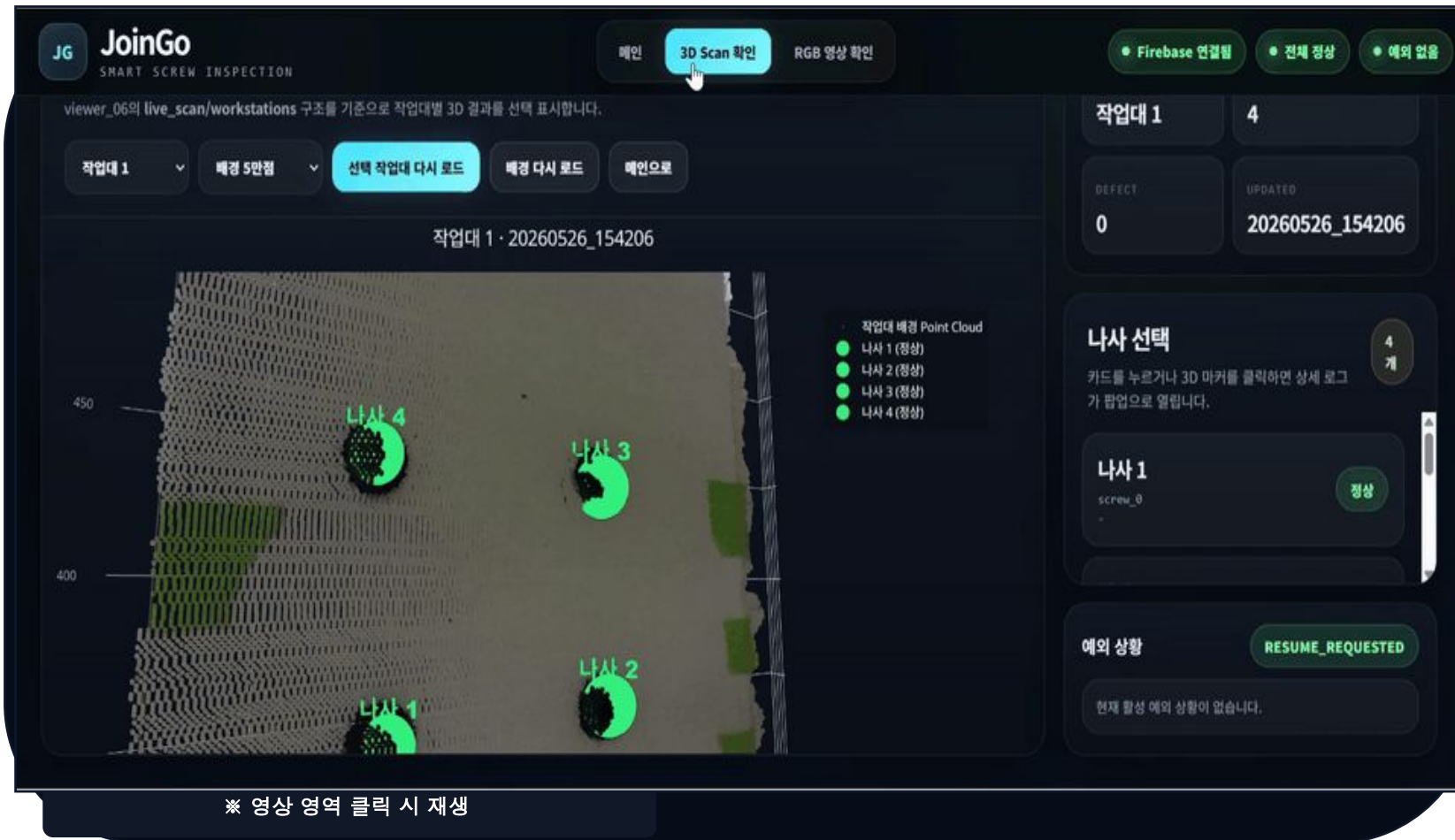
사용자 명령, 로봇 상태, AI 검사 결과, 3D 스캔 결과가 Firebase를 기준으로 하나의 화면에 모임



핵심: 웹은 모든 기능을 직접 수행하지 않고, DB 상태값을 기준으로 각 모듈의 결과를 같은 운영 화면에 정렬한다.

시연 영상 자료

최종 웹 대시보드에서 3D 검사 결과 선택, 나사별 정보 조회, 렉 완화 구조를 확인



영상 확인 포인트

작업대 선택 → 스캔 결과 로드
→ 나사별 상세 클릭 조회 순서로 웹 통합 기능을 확인한다.

발표 포인트

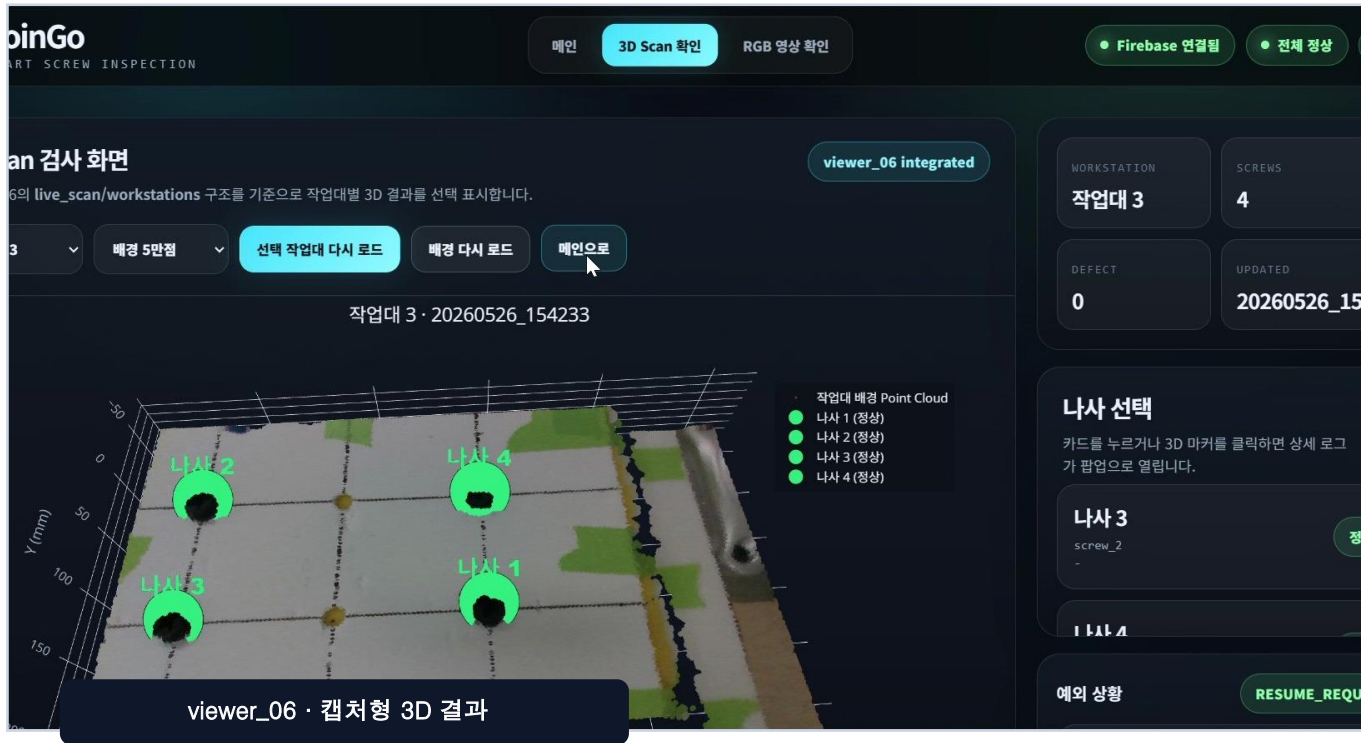
단순 UI 시연이 아니라 DB 기준 상태 동기화와 병목 완화가 실제 화면에 반영된 결과다.

기능 범위

렉 완화, 샘플링 레이트 선택, 나사별 상세 정보, 작업대별 스캔 결과 선택을 분리해 설명한다.

시연 기능 1 · 3D 렉 완화 구조

실시간 3D 스트리밍을 메인 웹에서 계속 갱신하지 않고, 완료된 캡처형 결과를 선택적으로 로드



문제

Point Cloud를 실시간으로 계속 반영하면 메인 웹 렌더링과 DB 상태 갱신이 동시에 밀려 렉이 발생했다.

해결

3d_scan_done 신호가 들어온 뒤 완성된 스캔 결과만 로드하는 캡처형 구조로 변경했다.

효과

메인 화면은 로봇 상태와 나사 상태를 유지하고, 3D 데이터는 필요 시점에만 불러 병목을 완화했다.

핵심 변화

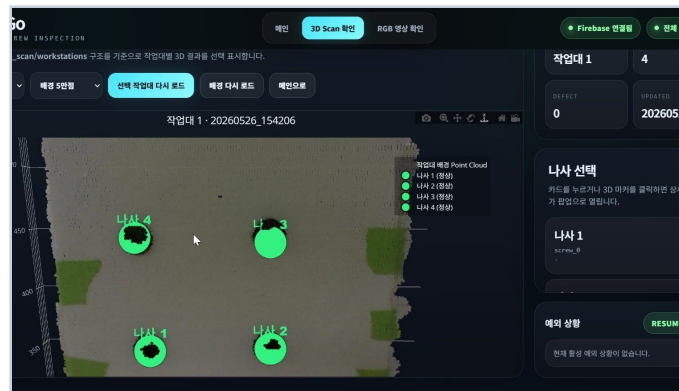
실시간 연속 갱신 중심 → 스캔 완료 후 캡처 결과 선택 로드 중심

시연 기능 2 · 샘플링 레이트 선택

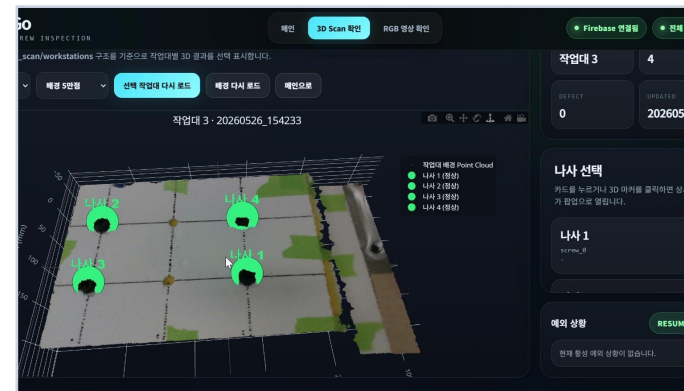
배경 Point Cloud 수를 1만~5만 점 범위에서 조절하여 성능과 가시성을 상황별로 선택



배경 1만점 · 성능 우선



배경 3만점 · 균형



배경 5만점 · 가시성 우선

낮은 샘플링

웹 반응성과 로딩 속도를 우선할 때 사용한다.
점군 수를 줄여 렌더링 부담을 낮춘다.

중간 샘플링

형상 파악과 성능 사이의 균형안이다.
일반 시연과 운영 화면에 적합하다.

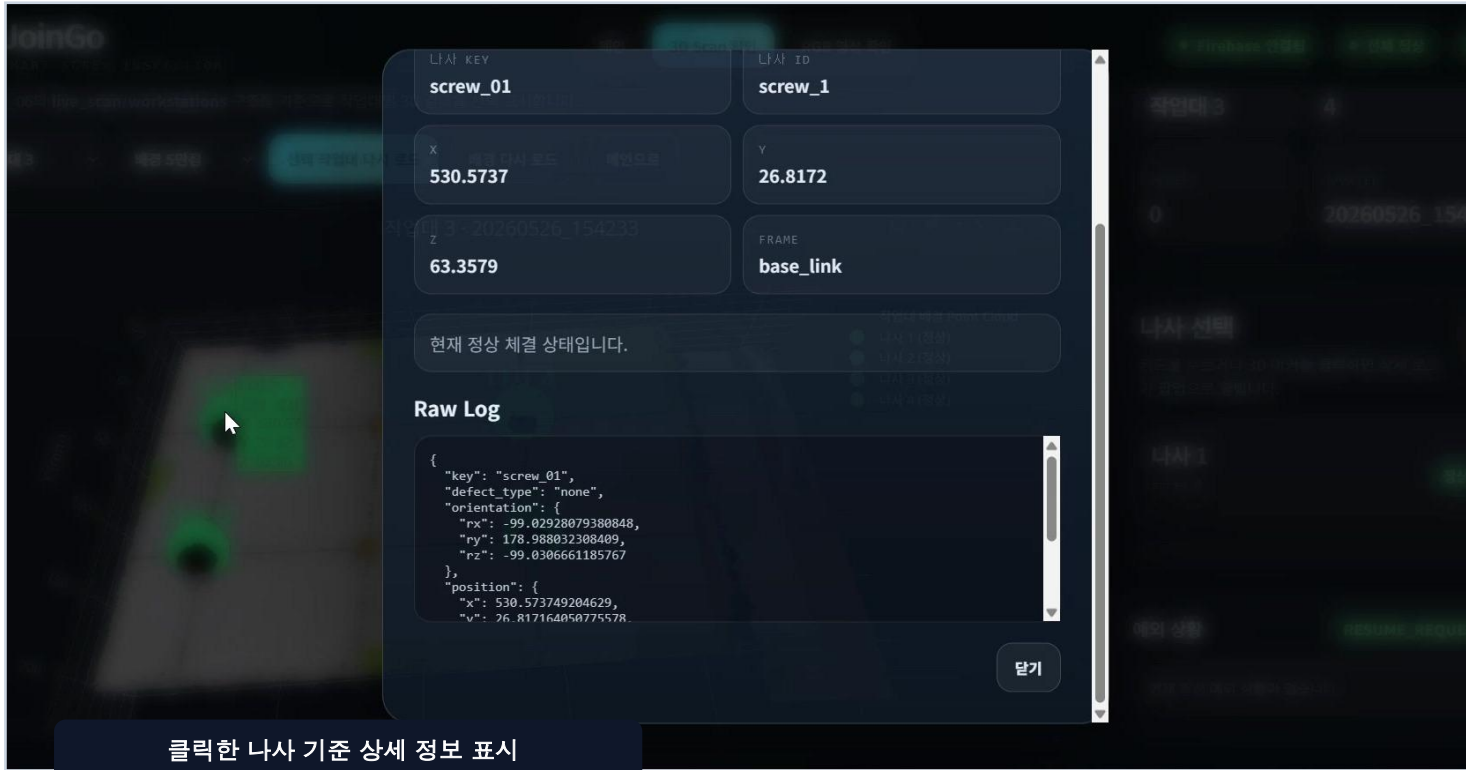
높은 샘플링

형상 확인이 중요할 때 선택한다.
색상은 유지해 부품·배경 판독성을 확보한다.

설계 의미: 3D 품질을 고정값으로 강제하지 않고, 상황별 병목 지점을 사용자가 직접 조절할 수 있게 했다.

시연 기능 3 · 나사별 정보 클릭 조회

3D 검사 화면의 마커 또는 우측 나사 카드를 클릭하면 좌표와 체결 상태를 즉시 확인



조회 대상

screw_key, screw_id, 상태값, 좌표 정보를 나사 단위로 분리 표시한다.

현장 의미

작업자가 불량 위치를 다시 찾지 않고 웹 화면에서 대응 대상을 바로 특정한다.

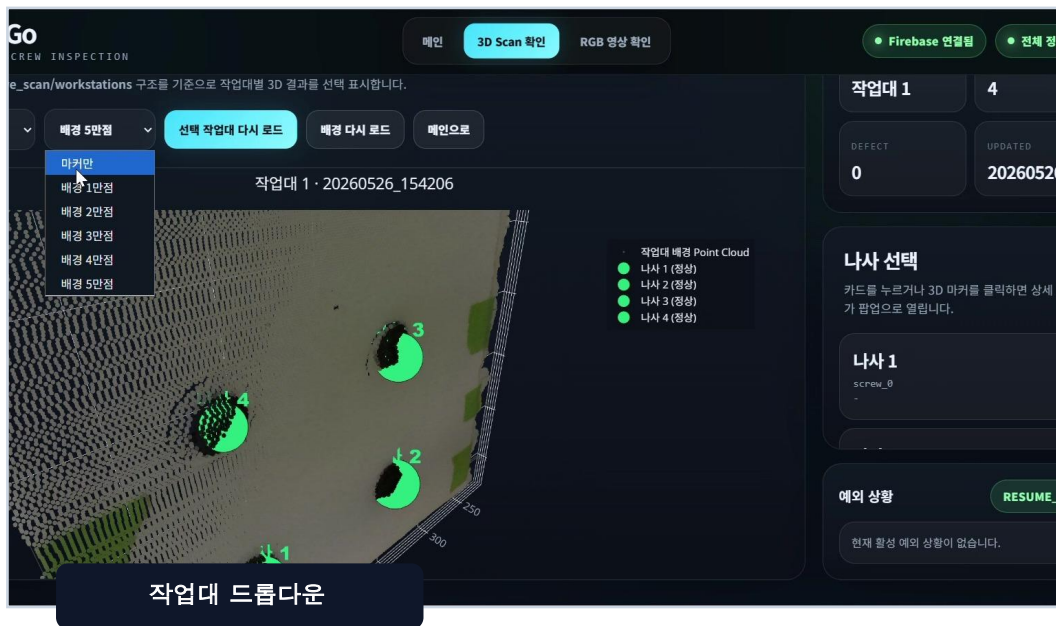
DB 의미

marker 단위 저장 구조와 연결되어 로봇 후속 작업·이력 조회에 재사용 가능하다.

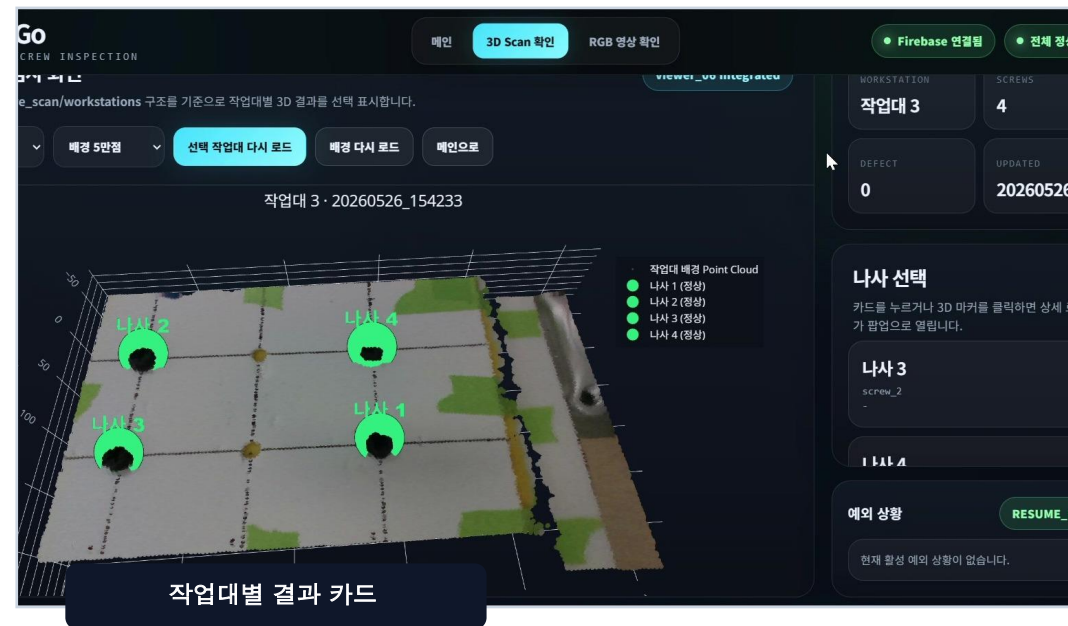
나사별 클릭 조회는 “3D 화면 표시”를 “작업 대상 선택 인터페이스”로 확장한 기능이다.

시연 기능 4 · 작업대별 스캔 결과 선택

여러 작업대의 3D 검사 결과를 동일 화면에서 선택해 확인하도록 viewer_06 구조를 메인 웹에 통합



작업대 드롭다운



작업대별 결과 카드

선택 구조

live_scan/workstations 기준으로 작업대 목록과 선택 결과를 분리한다.

운영 장점

작업대별 스캔 결과를 덮어쓰지 않고 비교·재확인할 수 있다.

통합 의미

로봇 제어 상태, 3D 결과, 나사 상태를 같은 화면에서 연결한다.

PART4 최종 성과와 남은 과제

웹 통합을 통해 자동 나사 체결 상태 점검 및 대응화 시스템의 운영 가능성을 높임

병목 완화

성과

실시간 3D 반영 대신 캡처형 결과 로드

선택적 품질 조절

성과

sample_rate로 속도와 가시성 조절

상태 통합

성과

로봇 제어 상태와 스캔 결과를 한 화면에 표시

시연 완성도

성과

검사 흐름을 작업자 관점으로 설명 가능

구분	현재 달성	향후 보완
성능	렉 유발 지점을 구조적으로 완화	FPS·로딩 시간·DB 지연 실측 필요
3D	완료 결과 중심으로 웹 표시 가능	좌표 정합성과 시점별 비교 기능 강화
로봇 통합	상태와 명령 흐름을 웹에 연결	오류 복구/안전 정지 UI 고도화

한 줄 요약: PART4 웹은 로봇·3D·DB를 “보여주는 화면”에서 “병목을 제어하고 운영 흐름을 통합하는 화면”으로 발전했다.

감사합니다

Data / Firebase DB / YOLO Debugging

Q&A

